Manisa Celal Bayar University

Faculty of Engineering — Department of Computer Engineering

Knowledge Engineering and Ontologies

2025–2026 Academic Year

FINAL PROJECT REPORT

Emotion-Aware Intelligent Recommendation Ontology

Team Members

Elif Özkanat — 220315023

Büşra Pehlivanlar — 220315051

Enes Ulucan — 220315018

Course Instructor

Dr. Gamze Türkmen

Submission Date: 15 June 2026

WIDOCO Documentation: elifozknt.github.io/emotion-aware-recommendation-ontology

GitHub Repository: github.com/elifozknt/emotion-aware-recommendation-ontology

Figma Design: https://steep-label-44397760.figma.site/

Table of Contents

1. Executive Summary

This project presents the design, implementation, and evaluation of the Emotion-Aware Intelligent Recommendation Ontology, a Semantic Web knowledge model that links a user's emotional state, personality profile, situational context, and behavioral tendencies to personalized digital content recommendations. The ontology was developed in OWL 2 / RDF (Turtle serialization) following the METHONTOLOGY methodology and is organized into 29 classes, 31 object properties, and 8 data properties, populated with approximately 70 individuals describing four sample users, five emotion types (organized into Positive, Negative, and Neutral sub-classes), needs, content items, effects, contexts, personality traits based on the Big Five model, platforms, feedback, behavior profiles, dynamic emotion sessions, user interactions, and data sources. The resulting knowledge graph was deployed in GraphDB, where its class hierarchy and inter-class relationships were visualized and explored. Eleven SPARQL queries were implemented and tested, directly addressing eight competency questions spanning basic retrieval, class-hierarchy reasoning, aggregation, temporal, and provenance-tracking scenarios. Data quality and structural integrity were verified using six SHACL shapes executed with pyshacl, yielding a result of Conforms: True. The ontology and supporting artifacts are documented through WIDOCO and published on GitHub, and a Figma-based prototype interface illustrates how ontology-driven recommendations could be surfaced to end users. Planned extensions include integrating Large Language Models for natural-language-to-SPARQL translation and LLM-assisted ontology population.

2. Description of the Project

2.1 Problem Statement and Motivation

Modern digital platforms — music streaming services, video-on-demand applications, podcast directories, and mobile wellness apps — recommend content primarily on the basis of historical behavior, collaborative filtering, or static demographic profiles. While effective for discovering items similar to past consumption, these approaches largely ignore a critical and highly transient factor that shapes what a person actually needs at a given moment: their current emotional state. A user who is stressed late at night, bored on a weekday evening, tired in the morning, or sad while with friends has fundamentally different content needs than what their long-term listening history would suggest. Existing recommender systems rarely model the chain that connects an emotion to the need it generates and, in turn, to the content that can satisfy that need, nor do they explicitly represent how personality traits, situational context (time of day, physical environment, social setting), and behavioral tendencies modulate the suitability of a recommendation.

The Emotion-Aware Intelligent Recommendation Ontology addresses this gap by providing a formal, machine-interpretable model of the Emotion → Need → Content → Effect recommendation chain, enriched with

Personality (Big Five), Context (time and environment), Platform availability, user Feedback, and — in its Phase 2 extension — temporally-tracked Dynamic Emotion, behavior profiling, session-based interaction logging, and data provenance. By representing this domain as an OWL 2 ontology and populating it as a knowledge graph, the project enables semantic querying (via SPARQL), automated consistency checking (via SHACL), and a foundation for future reasoning- and LLM-based recommendation services that can explain why a particular item was suggested.

2.2 Theoretical Foundations and Methodology

The ontology was engineered following the METHONTOLOGY methodology (Fernandez-Lopez, Gomez-Perez, & Juristo, 1997), which structures ontology development into specification, conceptualization, formalization, integration, implementation, and evaluation phases, supplemented by explicit versioning. In the specification phase, the domain scope (emotion-aware recommendation), intended uses (semantic querying and validation, future LLM-assisted population), and the competency questions listed in Section 2.3 were defined. In the conceptualization phase, the main concepts — User, Emotion, Need, Content, Recommendation, Effect, Context, Personality, Preference, Platform, Intention, and Feedback — and their relationships were sketched as a class-relationship diagram. The formalization phase implemented these concepts in OWL 2 using Protégé and exported them to RDF/Turtle. The integration phase incorporated design patterns from two published emotion-aware ontologies — the UniEmotion music-tagging ontology (Kim, 2013) and the movie recommendation ontology of Kim and Lee (2015) — to structure the Emotion sub-class hierarchy (Positive / Negative / Neutral) and the Content → Effect → Emotion relief chain. The implementation phase loaded the resulting ontology into a GraphDB repository to form an executable knowledge graph. Finally, the evaluation phase used SHACL shape validation and SPARQL competency-question testing to verify correctness and completeness, as detailed in Sections 6 and 7.

The project also draws on standard Semantic Web technologies throughout: RDF (Resource Description Framework) for representing triples, RDFS for basic class/property typing, OWL 2 for richer class axioms and property characteristics, SPARQL for declarative graph querying, and SHACL (Shapes Constraint Language) for declarative data validation. Dublin Core Terms, the VANN vocabulary, and Schema.org annotation properties were reused for ontology-level metadata (title, creator, license, version, code repository), following best practice for ontology interoperability.

2.3 Competency Questions

Competency questions (CQs) define the queries that the ontology and knowledge graph must be able to answer; they directly drove the class/property design described in Section 3 and were later implemented as

the SPARQL queries presented in Section 6. Table 1 lists the eight competency questions defined for this project and the SPARQL query (or queries) that address each one.

ID	Competency Question	Query
CQ1	Which emotional state is each user currently experiencing?	Q2
CQ2	Which users are currently experiencing a negative emotion (e.g., stress, sadness, tiredness)?	Q3
CQ3	Which content items are suitable for a user, given their personality type (e.g., introvert vs. extrovert)?	Q4
CQ4	Given a user's emotion, what need does that emotion create, and which content items satisfy that need?	Q5
CQ5	How many users fall into each high-level emotion category (Negative, Neutral, Positive)?	Q6
CQ6	How does a user's dynamic emotion intensity evolve over time, and which observations are the most intense?	Q7, Q11
CQ7	What interaction events (play, skip, complete) occurred with which content items during a user's emotion session?	Q8
CQ8	On which platforms is a given content item available, and what data source populated a given interaction record?	Q9, Q10

Table 1. Competency questions and their corresponding SPARQL queries.

In addition to the eight competency questions above, an initial repository sanity-check query (Q1) was implemented to confirm that the ontology and its instance data were loaded correctly into the GraphDB repository before competency-question testing began.

2.4 Expected Outputs, Tools, and Target Users

The expected outputs of the project are: (1) a validated OWL 2 / RDF Turtle ontology (ontology_v2.ttl) containing both the conceptual schema (TBox) and instance data (ABox); (2) a SHACL shapes file (shapes.ttl) together with a Python validation script (validate_shacl.py) and a validation report; (3) a GraphDB repository hosting the ontology as a queryable knowledge graph, including class hierarchy and class relationship visualizations; (4) a documented set of eleven SPARQL queries addressing the competency questions in Section 2.3; (5) WIDOCO-generated HTML documentation published via GitHub Pages; (6) a public GitHub repository containing all project artifacts; and (7) a Figma-based prototype user interface that illustrates how ontology-driven recommendations can be presented to an end user.

The tools and technologies used include Protégé for ontology authoring, GraphDB (Ontotext) as the triple store and SPARQL endpoint, Python with the pyshacl library for SHACL validation, WIDOCO (Wizard for Documenting Ontologies) for automatic documentation generation, and Figma for prototype design. The primary target users of the resulting system are consumers of digital content platforms (music, video, podcast,

and wellness applications) who would benefit from recommendations that adapt to their current emotional state, personality, and context. A secondary target audience is researchers and developers of emotion-aware recommender systems, who can reuse and extend the ontology schema — for example, by adding new emotion categories, content types, or behavior profiles — for their own applications.

3. Ontology Design

This section presents the conceptual model (TBox) and instance data (ABox) of the Emotion-Aware Intelligent Recommendation Ontology, describes the main classes, object properties, and data properties, and explains the modeling decisions taken during development.

3.1 Ontology Overview and Metadata

The ontology is published under the namespace http://example.org/emotion-aware-recommendation# with the preferred prefix ex:, serialized in RDF Turtle (.ttl) and exported from Protégé as OWL 2. Ontology-level metadata follows Dublin Core Terms (dc:), the VANN vocabulary (vann:), and Schema.org (schema:) conventions, recording the title, description, creators, publisher, creation and modification dates, license, and version history. Table 2 summarizes the key metadata fields of the current version (v2.0.0).

Property	Value
Ontology IRI / label	ex:EmotionAwareRecommendationOntology — “Emotion-Aware Intelligent Recommendation Ontology”
Namespace	http://example.org/emotion-aware-recommendation#
Preferred prefix	ex:
Serialization	OWL 2 / RDF (exported from Protégé), Turtle (.ttl)
Version	2.0.0 (priorVersion: 1.0.0)
License	Creative Commons Attribution 4.0 (CC BY 4.0)
Creators	Elif Özkanat, Büşra Pehlivanlar, Enes Ulucan
Publisher	Manisa Celal Bayar University
Created / Modified	2026-04-28 / 2026-05-14
Code repository	https://github.com/elifozknt/emotion-aware-recommendation-ontology
Total classes	29
Total object properties	31 (23 core + 8 Phase 2 extension)
Total data properties	8 (4 core + 4 Phase 2 extension)
Total individuals (ABox)	≈ 70

Table 2. Ontology metadata summary (v2.0.0).

3.2 TBox and ABox: Conceptual Schema vs. Instance Data

Following standard ontology engineering terminology, the ontology is divided into a TBox (Terminological Box) and an ABox (Assertional Box). The TBox defines the conceptual schema of the domain: 29 classes (concepts), the subclass hierarchy among them, 31 object properties (relationships between individuals of different classes), 8 data properties (relationships between individuals and literal values), and the rdfs:domain / rdfs:range restrictions placed on each property. The ABox consists of approximately 70 concrete individuals that instantiate these classes — four sample users (Elif, Büsra, Enes, Deniz), five emotion individuals (Stress, Sadness, Anxiety, Tiredness, Boredom), six content items, five needs, five effects, six personality individuals, four behavior profiles, three dynamic-emotion observations, one emotion session, two user interactions, two data sources, and a set of recommendations, feedback records, preferences, platforms, intentions, content categories, and contextual individuals (time contexts and environments). Every individual in the ABox is asserted to be rdf:type of exactly one TBox class (or, for personality and emotion sub-types, a leaf sub-class), and is connected to other individuals through the object properties defined in the TBox.

3.3 Class Hierarchy (TBox)

The ontology defines 29 classes organized around twelve top-level concepts, four of which (Emotion, Context, and Personality, plus the newly introduced Phase 2 top-level classes) have explicit sub-class hierarchies. Table 3 lists all 29 classes together with their superclass (where applicable) and a short description of the role each class plays in the recommendation model.

Class	Superclass	Description
ex:User	—	A person who receives recommendations from the system.
ex:Emotion	—	An emotional state experienced by a user; categorized as Positive, Negative, or Neutral (UniEmotion-inspired).
ex:PositiveEmotion	ex:Emotion	Pleasant affective states (e.g., happiness, excitement); reserved for future population.
ex:NegativeEmotion	ex:Emotion	Unpleasant affective states. Members: Stress, Sadness, Anxiety, Tiredness.
ex:NeutralEmotion	ex:Emotion	Factual / low-valence affective states. Member: Boredom.
ex:DynamicEmotion	ex:Emotion	A temporally-tracked emotion state with an intensity level, recorded within an EmotionSession.
ex:Need	—	A user need that may arise from an emotional state (e.g., Relaxation, Motivation).
ex:Content	—	A digital content item or activity that can be recommended to a user.
ex:Recommendation	—	A personalized suggestion generated for a user.

Class	Superclass	Description
ex:Effect	—	The expected psychological or behavioral effect of a content item.
ex:Context	—	The situation in which the user is located.
ex:TimeContext	ex:Context	The time-related context of a user (Morning, Evening, Night).
ex:Environment	ex:Context	The physical or social environment of a user (Home, School, Alone, With Friends).
ex:Personality	—	A Big-Five-based personality trait that may affect recommendation suitability.
ex:IntrovertedPersonality	ex:Personality	Preference for calm, solo, low-stimulation content.
ex:ExtrovertedPersonality	ex:Personality	Preference for social, energetic, group-oriented content.
ex:AgreeablePersonality	ex:Personality	Preference for emotionally warm, cooperative content.
ex:NeuroticPersonality	ex:Personality	Preference for calming, grounding content (stress-sensitive users).
ex:OpennessPersonality	ex:Personality	Preference for novel, artistic, experimental content.
ex:ConscientiousPersonality	ex:Personality	Preference for structured, educational, purposeful content.
ex:Preference	—	A user's preferred content type or activity.
ex:ContentCategory	—	A category describing the type of content (Music, Movie, Podcast, Meditation, Game).
ex:Platform	—	A digital platform where content is delivered (Spotify, Netflix, Mobile App).
ex:Intention	—	The intended purpose of a recommendation (e.g., Reduce Stress).
ex:Feedback	—	A user's response to a recommendation.
ex:BehaviorProfile	—	Classification of a user's interaction behavior style (Cognitive, Speed, Sedentary, Aggressive).
ex:EmotionSession	—	A bounded time window during which a user's emotion and interactions are recorded.
ex:UserInteraction	—	A single recorded interaction event between a user and recommended content (play/skip/complete).
ex:DataSource	—	A provenance record for data used to populate ontology instances (e.g., Spotify API, Kaggle dataset).

Table 3. The 29 classes of the Emotion-Aware Intelligent Recommendation Ontology.

3.3.1 Class Hierarchy Visualization (GraphDB)

Figure 1 shows the overall class hierarchy as visualized in GraphDB's Class Hierarchy view. Each circle represents a class, sized according to the number of instances and relationships associated with it; the outer

ring groups sub-classes under their parent class. The diagram confirms that the repository contains the expected 29 classes.

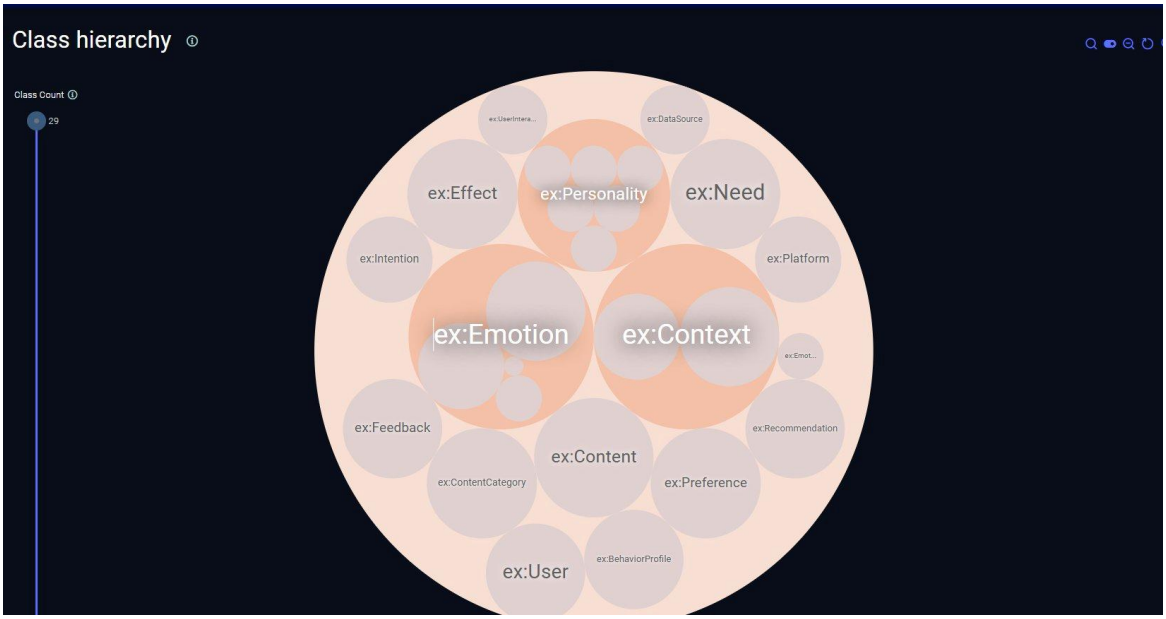

Figure 1. GraphDB Class Hierarchy view showing all 29 classes (Class Count = 29).

Figures 2 and 3 show detailed views of two sub-class groupings that were central to the Phase 2 extension: the six Personality sub-classes derived from the Big Five model, and the Negative / Neutral / Dynamic Emotion sub-classes derived from the UniEmotion ontology (Kim, 2013).

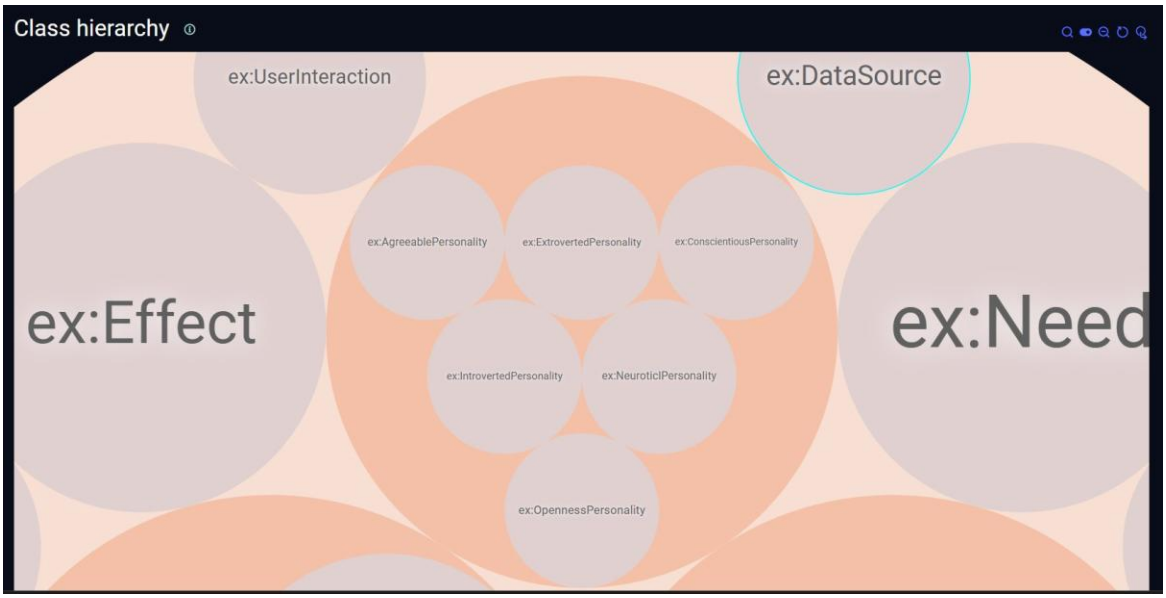

Figure 2. Detail view: the six Personality sub-classes (Agreeable, Extroverted, Conscientious, Introverted, Neurotic, Openness) nested under ex:Personality, alongside ex:UserInteraction and ex:DataSource.

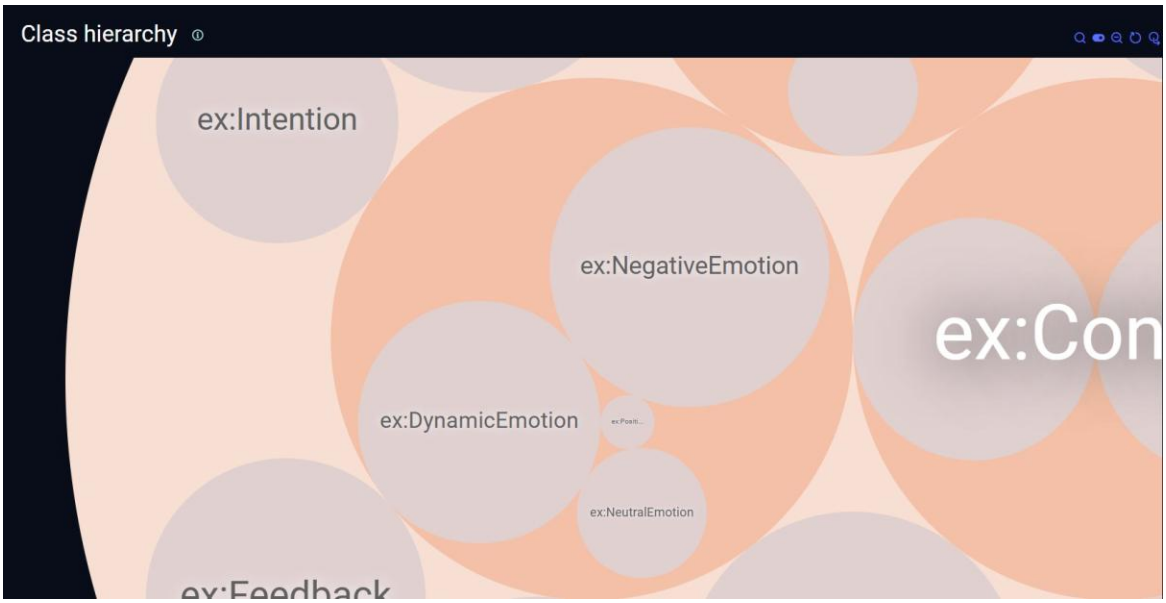

Figure 3. Detail view: ex:NegativeEmotion, ex:NeutralEmotion, ex:PositiveEmotion and ex:DynamicEmotion nested under ex:Emotion, alongside ex:Intention and ex:Feedback.

3.4 Object Properties (TBox)

The ontology defines 31 object properties that connect individuals across classes, encoding the recommendation logic (Emotion → Need → Content → Effect), contextual and personality matching, and — in the Phase 2 extension — temporal emotion tracking, behavior profiling, session-based interaction logging, and data provenance. Each object property declares an rdfs:domain and rdfs:range. Table 4 lists the 23 core (Phase 1) object properties, and Table 5 lists the 8 object properties introduced in the Phase 2 extension.

Property	Domain	Range	Description
ex:feels	User	Emotion	Connects a user to an emotion they experience.
ex:createsNeed	Emotion	Need	Connects an emotion to the need it may create.
ex:hasNeed	User	Need	Connects a user to a need.
ex:satisfiedBy	Need	Content	Connects a need to content that can satisfy it.
ex:hasEffect	Content	Effect	Connects content to its expected effect.
ex:relievesEmotion	Effect	Emotion	Connects an effect to an emotion it may help reduce.
ex:belongsToCategory	Content	ContentCategory	Connects content to its category.
ex:hasPreference	User	Preference	Connects a user to a preference.
ex:matchesPreference	Content	Preference	Connects content to a matching user preference.
ex:hasPersonality	User	Personality	Connects a user to a personality trait.
ex:suitableForPersonality	Content	Personality	Shows which personality type a content item is suitable for.
ex:hasTimeContext	User	TimeContext	Connects a user to a time context.
ex:isInEnvironment	User	Environment	Connects a user to an environment.

Property	Domain	Range	Description
ex:suitableForTime	Content	TimeContext	Shows which time context is suitable for content.
ex:suitableForEnvironment	Content	Environment	Shows which environment is suitable for content.
ex:givenTo	Recommendation	User	Connects a recommendation to the user receiving it.
ex:includesContent	Recommendation	Content	Connects a recommendation to the content it includes.
ex:basedOnEmotion	Recommendation	Emotion	Shows the emotion considered in a recommendation.
ex:basedOnNeed	Recommendation	Need	Shows the need considered in a recommendation.
ex:considersContext	Recommendation	Context	Shows contextual information used in a recommendation.
ex:hasIntention	Recommendation	Intention	Shows the intended purpose of a recommendation.
ex:availableOnPlatform	Content	Platform	Connects content to the platform where it is available.
ex:hasFeedback	Recommendation	Feedback	Connects a recommendation to user feedback.

Table 4. Core (Phase 1) object properties (23 total).

Property	Domain	Range	Description
ex:hasDynamicEmotion	User	DynamicEmotion	Connects a user to a temporally-tracked dynamic emotion state.
ex:hasBehaviorProfile	User	BehaviorProfile	Connects a user to their predicted behavior profile.
ex:recordedInSession	DynamicEmotion	EmotionSession	Links a dynamic emotion observation to the session in which it was recorded.
ex:hasInteraction	EmotionSession	UserInteraction	Connects an emotion session to individual interaction events within it.
ex:interactedWith	UserInteraction	Content	Connects an interaction event to the content item the user acted on.
ex:changesOverTime	DynamicEmotion	DynamicEmotion	Links two dynamic emotion states to represent a temporal transition.
ex:populatedFrom	UserInteraction	DataSource	Documents the data source from which an interaction instance was extracted.
ex:derivedFromContext	DynamicEmotion	Context	Links a dynamic emotion state to the contextual factors that contributed to it.

Table 5. Phase 2 extension object properties (8 total).

3.5 Data Properties (TBox)

Eight data properties connect individuals to literal (XSD-typed) values. Table 6 lists all data properties, four of which were introduced in the Phase 2 extension to support intensity scoring, session timestamps, interaction typing, and source URLs.

Property	Domain	Range	Description
ex:hasAge	User	xsd:integer	The user's age in years.
ex:hasConfidenceScore	Recommendation	xsd:decimal	A confidence score (0.0–1.0) for a recommendation.
ex:hasDurationMinutes	Content	xsd:integer	The typical duration of a content item, in minutes.
ex:hasFeedbackScore	Feedback	xsd:integer	A numeric rating given as feedback (e.g., 1–5).
ex:hasIntensityLevel	DynamicEmotion	xsd:decimal	A numeric score (0.0–1.0) representing the intensity of a dynamic emotion.
ex:hasTimestamp	EmotionSession	xsd:dateTime	The date-time when the emotion session was recorded.
ex:hasInteractionType	UserInteraction	xsd:string	The type of interaction: 'play', 'skip', or 'complete'.
ex:hasSourceURL	DataSource	xsd:anyURI	The URL of the data source used for ontology population.

Table 6. Data properties (8 total; the last four were introduced in the Phase 2 extension).

3.6 Modeling Decisions and Design Rationale

3.6.1 Classes vs. Instances

A recurring design decision was whether a concept should be represented as a class (a category with potential multiple members and further sub-types) or as an individual (a single named instance). General, extensible categories — emotions, personality dimensions, content categories, behavior profiles — were modeled as classes (or sub-classes of a parent class) whenever the category itself could plausibly have multiple distinguishable members that participate differently in the ontology's relationships. For example, Personality was modeled as a class with six Big-Five-inspired sub-classes (IntrovertedPersonality, ExtrovertedPersonality, AgreeablePersonality, NeuroticPersonality, OpennessPersonality, ConscientiousPersonality), each populated by a single representative individual (Introvert, Extrovert, AgreeableUser, NeuroticUser, OpenUser, ConscientiousUser). This allows new personality types to be added as additional sub-classes without changing the schema, and allows SPARQL queries to reason over the Personality hierarchy generically (e.g., "retrieve content suitable for any sub-class of Personality"). In contrast, concrete, non-extensible values — specific time slots (Morning, Evening, Night), specific platforms (Spotify, Netflix, MobileApp), or specific content items (LoFiPlaylist, ComfortMovie) — were modeled as individuals of a small, fixed set of classes (TimeContext, Platform, Content), since these represent enumerable members of a finite catalog rather than open-ended categories.

3.6.2 Emotion Sub-Class Hierarchy (UniEmotion-Inspired)

The Emotion class is specialized into three sub-classes — PositiveEmotion, NegativeEmotion, and NeutralEmotion — directly inspired by the three-way tag categorization (positive emotional tags, negative emotional tags, and factual tags) used in the UniEmotion music recommendation ontology (Kim, 2013). All five emotion individuals used in the ABox (Stress, Sadness, Anxiety, Tiredness, Boredom) are typed as instances

of NegativeEmotion or NeutralEmotion rather than the generic Emotion class; PositiveEmotion remains an empty class ready for future population (e.g., Happiness, Excitement) as the user base grows. This polarity-based sub-typing enables aggregate, polarity-aware queries such as Q3 (retrieve users with any negative emotion) and Q6 (count users per emotion category) without enumerating individual emotions in the query itself — a direct benefit of representing polarity as class membership rather than as a separate data property.

3.6.3 Dynamic Emotion, Sessions, and Temporal Modeling

Phase 1 modeled emotion as a static snapshot: each user feels exactly one emotion at a time, asserted via ex:feels. Phase 2 introduces ex:DynamicEmotion as a sub-class of Emotion to capture how a user's emotional intensity evolves across an interaction session. Each DynamicEmotion individual carries a ex:hasIntensityLevel value (a decimal between 0.0 and 1.0), is linked to the contextual factors that produced it via ex:derivedFromContext, and may be linked to an earlier DynamicEmotion state via ex:changesOverTime, representing a temporal transition (e.g., ElifStressLow ex:changesOverTime ElifStressHigh). The ex:EmotionSession class groups a set of ex:UserInteraction events that occurred within a bounded time window (recorded via ex:hasTimestamp), and DynamicEmotion observations are linked to the session in which they occurred via ex:recordedInSession. This design was informed by the temporal/sequential dependency modeling used in the GRU-RNN behavioral prediction framework of Fernandez et al. (2023): rather than representing a full sequence-model architecture in the ontology, the key insight — that a user's state evolves across discrete observations linked in sequence — was captured declaratively using the changesOverTime shortcut property, which simplifies a longer role chain into a single directly-traversable relation and improves both human readability and SPARQL query simplicity.

3.6.4 Behavior Profiles, User Interactions, and Provenance

The BehaviorProfile class (with four individuals: CognitiveBehavior, SpeedBehavior, SedentaryBehavior, AggressiveBehavior) represents a coarse classification of how a user tends to interact with recommended content, conceptually adapted from the four behavior classes predicted by the GRU-RNN / Deep-Training-Tree model in Fernandez et al. (2023). Rather than implementing the predictive model itself, the ontology represents its output as a controlled vocabulary that a future classifier could populate via ex:hasBehaviorProfile, enabling downstream recommendation logic to adjust content length and novelty per user (e.g., a SpeedBehavior user such as Elif is more likely to be shown short-form content such as the five-minute BreathingExercise).

The UserInteraction class records individual play / skip / complete events (ex:hasInteractionType) and links them to the content acted upon (ex:interactedWith). Each interaction additionally carries an ex:populatedFrom link to a DataSource individual (e.g., SpotifyAPISource), explicitly recording the provenance of automatically-populated instances. This design follows the data-provenance recommendation of Norouzi

et al. (2025) for LLM-assisted ontology population: by recording which source produced each instance, future quality-auditing or re-extraction processes can selectively re-validate or re-populate instances by source.

3.6.5 Part-Whole and Role-Chain Relationships

Several relationships in the ontology represent part-whole or aggregation structures rather than strict subsumption. An EmotionSession is composed of multiple UserInteraction individuals (ex:hasInteraction), and a Recommendation aggregates references to the Emotion, Need, Context, Content, Intention, and Feedback that justify it (ex:basedOnEmotion, ex:basedOnNeed, ex:considersContext, ex:includesContent, ex:hasIntention, ex:hasFeedback). Rather than modeling Recommendation as a single object with embedded sub-objects (which OWL does not support directly), these aggregations are represented as multiple object properties from the Recommendation individual to the relevant component individuals — an explicit "hub" pattern in which Recommendation connects otherwise unrelated individuals (a User, an Emotion, a Need, several Context values, a Content item, an Intention, and a Feedback record) for a single recommendation event. This pattern keeps the schema simple (no reified n-ary relations are required) while still allowing every component of a recommendation's justification to be retrieved through a single SPARQL query (see Q5).

4. Data Acquisition

4.1 Data Acquisition Strategy

The ABox of the Emotion-Aware Intelligent Recommendation Ontology was populated through expert-curated instance data: the three ontology engineers manually authored a representative set of individuals — four sample users (Elif, Büsra, Enes, Deniz) with associated emotions, needs, personalities, preferences, and contexts; a catalogue of six content items spanning five content categories; and the recommendation, feedback, behavior-profile, dynamic-emotion, session, and interaction individuals described in Section 3 — directly in RDF Turtle, following the class and property definitions established during the conceptualization phase. This expert-curation approach ensured that every instance is correctly typed, satisfies the SHACL shapes presented in Section 7, and exercises every object and data property defined in the TBox at least once, which is essential for demonstrating the competency questions in Section 6.

Although the current ABox was populated manually, the ontology was explicitly designed to support automated, large-scale population from external data sources in future phases. To this end, the Phase 2 extension introduces the ex:DataSource class and the ex:populatedFrom object property, which record the provenance of an instance — i.e., which external source it would be derived from. Table 7 summarizes the external data sources that were studied and cataloged to support this future population pipeline; only the

Spotify Web API and the Kaggle Music-Emotion dataset are currently represented as DataSource individuals in the ABox (ex:SpotifyAPISource and ex:KaggleEmotionDataset), while the remaining sources informed the schema design but are not yet instantiated.

Source	Type	Role in This Project
Spotify Web API	REST API	Reference source for user listening-history fields (track ID, timestamp, duration, audio features). Represented in the ABox as ex:SpotifyAPISource and linked via ex:populatedFrom from the two illustrative UserInteraction individuals (Interaction001, Interaction002).
Kaggle: Music & Emotion Recognition	Public dataset (CSV)	Reference source for mapping audio features (valence, energy, tempo) to Effect / Emotion categories. Represented in the ABox as ex:KaggleEmotionDataset.
GoEmotions (Google / Kaggle)	Public dataset	Considered as a future source of fine-grained, 27-category text-based emotion labels for free-text emotion detection; not yet instantiated as a DataSource individual.
Structured user survey	Primary data (planned)	Designed instrument for collecting demographics, Big-Five personality self-assessment, content-category preferences, and typical context (time of day, environment), to be mapped to User, Personality, Preference, TimeContext, and Environment individuals.
Expert / manual annotation	Domain knowledge	Used to author the Emotion → Need → Content → Effect mappings, the personality-to-content suitability links, and all instance data currently in the ABox.

Table 7. Data sources studied for ontology population, and their current role in the project.

4.2 Data Format and Preprocessing

All instance data is represented directly as RDF triples in Turtle syntax within ontology_v2.ttl; no intermediate tabular files are produced for the current ABox, since the data was authored directly in RDF rather than transformed from an external tabular source. For the two DataSource-linked example interactions, the design follows the preprocessing pipeline that a future, larger-scale population process would apply to raw Spotify API responses or Kaggle CSV rows:

- JSON flattening: nested API responses (e.g., /me/player/recently-played) would be flattened into rows with the fields user_id, track_id, timestamp, duration_ms, and context_type.
- Label normalization: free-text emotion labels would be mapped onto the controlled vocabulary defined by the NegativeEmotion / NeutralEmotion / PositiveEmotion sub-classes.
- Interaction-type classification: events would be classified as ‘play’ (started), ‘skip’ (duration below a threshold), or ‘complete’ (duration above a threshold), populating ex:hasInteractionType — the same three values enforced by the SHACL sh:in constraint described in Section 7.

- Timestamp normalization: all timestamps would be converted to ISO 8601 xsd:dateTime, matching the format used for ex:hasTimestamp (e.g., "2026-05-14T21:30:00").
- Missing-value handling: records with implausibly short durations or incomplete survey responses would be excluded prior to RDF generation.

This preprocessing design ensures that any future automated population — whether script-based ETL or LLM-assisted extraction following Norouzi et al. (2025) — would produce instances directly compatible with the existing schema and SHACL shapes, without further transformation.

4.3 Mapping Data Concepts to Ontology Elements

Table 8 shows how each conceptual data field maps to an ontology class or property, together with the example individuals currently present in the ABox that instantiate that mapping.

Conceptual Data Field	Ontology Class / Property	Example Individual(s) in ABox
User identifier	ex:User	ex:Elif, ex:Busra, ex:Enes, ex:Deniz
Negative emotion label	ex:NegativeEmotion via ex:feels	ex:Stress, ex:Sadness, ex:Anxiety, ex:Tiredness
Neutral emotion label	ex:NeutralEmotion via ex:feels	ex:Boredom
Content / track identifier	ex:Content	ex:LoFiPlaylist, ex:BreathingExercise, ex:ComfortMovie, etc.
Audio valence feature	ex:Effect via ex:hasEffect	ex:CalmingEffect, ex:RelaxingEffect, ex:MotivatingEffect
Session time window	ex:EmotionSession via ex:hasTimestamp	ex:Session001 (2026-05-14T21:30:00)
Interaction event	ex:UserInteraction via ex:hasInteractionType	ex:Interaction001 (play), ex:Interaction002 (complete)
Context (time / place)	ex:TimeContext / ex:Environment	ex:Night, ex:Home, ex:Alone, ex:School, ex:WithFriends
Personality self-report	ex:Personality sub-class via ex:hasPersonality	ex:Introvert, ex:Extrovert, ex:AgreeableUser, ex: NeuroticUser, ex:OpenUser, ex:ConscientiousUser
Content preference	ex:Preference via ex:hasPreference	ex:LikesMusic, ex:LikesPodcasts, ex:LikesGames, ex:LikesMovies, ex:LikesShortContent
Data provenance	ex:DataSource via ex:populatedFrom	ex:SpotifyAPISource, ex:KaggleEmotionDataset
Predicted behavior class	ex:BehaviorProfile via ex:hasBehaviorProfile	ex:SpeedBehavior (Elif), ex:CognitiveBehavior (Enes)

Table 8. Mapping between conceptual data fields and ontology classes / properties.

4.4 Data Quality and Limitations

Because the current ABox was expert-curated rather than derived from a large external dataset, it is internally consistent by construction and fully satisfies the SHACL shapes defined in Section 7 (Conforms: True). However, its scale is intentionally small — four users, six content items, and a single emotion session — which is sufficient to demonstrate every class, property, and competency question, but not representative of a production-scale recommender system. The two ex:populatedFrom links to ex:SpotifyAPISource document the intended provenance pattern for future API-derived data but do not reflect a live API connection in the current prototype. Likewise, the ex:PositiveEmotion class and the GoEmotions-derived emotion categories remain unpopulated, representing planned extensions rather than current limitations of the schema itself. These points are revisited as concrete future-work items in Section 10.

5. Knowledge Graph Construction

This section describes how the ontology (TBox) and the instance data (ABox) presented in Section 3 were combined and deployed as a queryable knowledge graph in GraphDB, illustrates representative RDF triples drawn from the ABox, and presents the system architecture connecting the knowledge graph to the SPARQL query layer, the SHACL validation pipeline, the WIDOCO documentation, and ... the Figma-based prototype interface.

5.1 RDF Triple Model

The knowledge graph is represented as a set of RDF triples, each consisting of a subject, a predicate, and an object, written in Turtle syntax under the ex: namespace (http://example.org/emotion-aware-recommendation#). Class membership is asserted via rdf:type, sub-class relationships via rdfs:subClassOf, object-property relationships link two individuals (subject and object are both IRIs), and data-property relationships link an individual to a typed literal (e.g., xsd:decimal, xsd:dateTime, xsd:string, xsd:integer). Every individual in the ABox carries an rdfs:label (with an @en language tag) for human-readable display in the prototype interface when rendering recommendation cards.

5.2 Representative RDF Triples

Listing 1 shows a representative subset of triples drawn directly from ontology_v2.ttl, illustrating the full Emotion → Need → Content → Effect reasoning chain for the user Elif, her personality-based content match, her Phase 2 dynamic-emotion transition, and the session / interaction / provenance chain that grounds Interaction001.

```
# --- User, emotion, need, content, effect chain ---
ex:Elif        rdf:type            ex:User .
ex:Elif        ex:feels            ex:Stress .
ex:Stress      rdf:type            ex:NegativeEmotion .
ex:Stress      ex:createsNeed      ex:Relaxation .
ex:Relaxation  ex:satisfiedBy      ex:LoFiPlaylist .
ex:LoFiPlaylist ex:hasEffect       ex:CalmingEffect .
ex:CalmingEffect ex:relievesEmotion ex:Stress .

# --- Personality-based matching ---
ex:Elif        ex:hasPersonality        ex:Introvert .
ex:Introvert   rdf:type                 ex:IntrovertedPersonality .
ex:LoFiPlaylist ex:suitableForPersonality ex:Introvert .

# --- Phase 2: dynamic emotion and temporal transition ---
ex:Elif         ex:hasDynamicEmotion ex:ElifStressHigh .
ex:ElifStressHigh  ex:hasIntensityLevel  0.85 .
ex:ElifStressLow   ex:hasIntensityLevel  0.30 .
ex:ElifStressLow   ex:changesOverTime    ex:ElifStressHigh .

# --- Phase 2: session, interaction, and provenance ---
ex:Session001    ex:hasTimestamp "2026-05-14T21:30:00"^^xsd:dateTime .
ex:Session001    ex:hasInteraction    ex:Interaction001 .
ex:Interaction001 ex:hasInteractionType "play" .
ex:Interaction001 ex:interactedWith   ex:LoFiPlaylist .
ex:Interaction001 ex:populatedFrom    ex:SpotifyAPISource .
ex:ElifStressHigh ex:recordedInSession ex:Session001 .

# --- Recommendation as a hub node ---
ex:Recommendation001 ex:givenTo          ex:Elif .
ex:Recommendation001 ex:includesContent  ex:LoFiPlaylist .
ex:Recommendation001 ex:basedOnEmotion   ex:Stress .
ex:Recommendation001 ex:basedOnNeed      ex:Relaxation .
ex:Recommendation001 ex:hasConfidenceScore 0.92 .
```

Listing 1. Representative RDF triples (Turtle syntax) illustrating the Emotion → Need → Content → Effect chain, personality matching, dynamic emotion transition, session/interaction logging, and the Recommendation hub pattern.

This excerpt demonstrates how a single user (Elif) is connected, through nine distinct object properties, to twelve other individuals spanning seven different classes — a level of interconnection that is characteristic of the knowledge graph as a whole and is quantitatively confirmed by the class-relationship analysis in Section 5.4.

5.3 GraphDB Deployment

The ontology and its instance data were deployed as a knowledge graph using GraphDB (Ontotext), a native RDF triple store with a built-in SPARQL 1.1 endpoint and a graphical workbench for exploration and querying. The deployment process consisted of the following steps:

- A new local repository was created in the GraphDB Workbench (Setup → Repositories → Create new repository), configured with RDFS reasoning enabled so that class-hierarchy-aware queries (e.g.,

retrieving all individuals typed as a sub-class of ex:NegativeEmotion or ex:Personality) return correct results without requiring explicit rdf:type assertions for every super-class.

- The file ontology_v2.ttl was imported via Import → RDF → Upload RDF files, using Turtle as the source format and the ontology's own namespace (http://example.org/emotion-aware-recommendation#) as the base IRI; this loaded both the TBox (29 classes, 31 object properties, 8 data properties) and the ABox (≈ 70 individuals) into the same named graph.
- The ex: prefix was registered in the repository's namespace list so that all SPARQL queries (Section 6) and the Class Hierarchy / Class Relationships views (below) display human-readable, prefixed identifiers instead of full IRIs.
- Successful loading was confirmed by running the repository sanity-check query Q1 (SELECT * WHERE { ?s ?p ?o } LIMIT 100), which returned 100 triples drawn from across the TBox and ABox, confirming that the repository was populated correctly before competency-question testing began.
- GraphDB's built-in Class Hierarchy and Class Relationships visualizations (Explore → Class hierarchy / Class relationships) were used to visually verify the structure of the loaded knowledge graph, as shown in Figure 1 (Section 3.3.1) and Figure 4 (below).

5.4 Knowledge Graph Visualization: Class Relationships

Figure 4 shows GraphDB's Class Relationships (chord diagram) view, which visualizes the dependency links between the ten most-connected classes in the knowledge graph. Each arc segment represents a class, and each ribbon represents the aggregate number of triples whose subject is typed as one class and whose object is typed as the other. The accompanying table reports the total number of incoming and outgoing links per class.

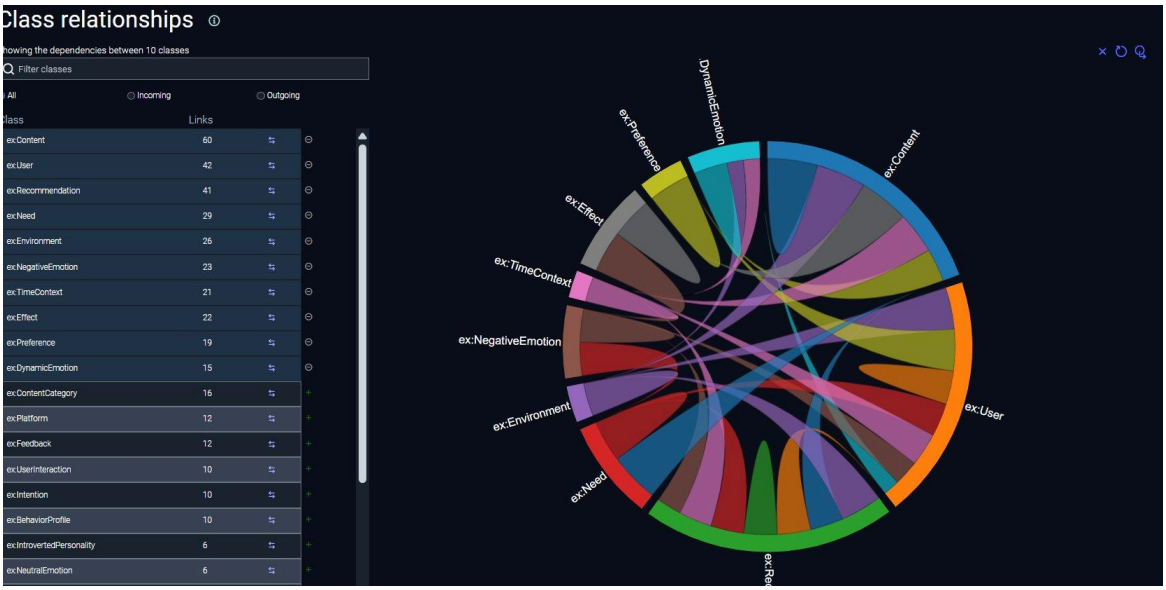

Figure 4. GraphDB Class Relationships view (chord diagram) showing the ten most-connected classes and their link counts.

As Table 9 summarizes, ex:Content (60 links) and ex:User (42 links) are by far the most heavily connected classes, followed by ex:Recommendation (41) and ex:Need (29). This is consistent with the ontology's design intent: Content is the target of the recommendation chain and is connected via ex:hasEffect, ex:belongsToCategory, ex:matchesPreference, ex:suitableForPersonality, ex:suitableForTime, ex:suitableForEnvironment, ex:availableOnPlatform, and ex:interactedWith, while User is the source of the chain via ex:feels, ex:hasNeed, ex:hasPersonality, ex:hasPreference, ex:hasTimeContext, ex:isInEnvironment, ex:hasDynamicEmotion, and ex:hasBehaviorProfile. The high connectivity of ex:Recommendation (41 links) reflects its role as the "hub" individual described in Section 3.6.5, aggregating six different relationships per recommendation.

Class	Links	Class	Links
ex:Content	60	ex:Effect	22
ex:User	42	ex:Preference	19
ex:Recommendation	41	ex:DynamicEmotion	15
ex:Need	29	ex:ContentCategory	16
ex:Environment	26	ex:Platform	12
ex:NegativeEmotion	23	ex:Feedback	12
ex:TimeContext	21	ex:UserInteraction / ex:Intention / ex:BehaviorProfile	10 each

Table 9. Class connectivity (incoming + outgoing links) as reported by the GraphDB Class Relationships view.

5.5 System Architecture and Prototype Interface

Figure 5 summarizes the end-to-end architecture connecting ontology engineering, the GraphDB knowledge graph, the validation and documentation pipeline, and the prototype user interface.

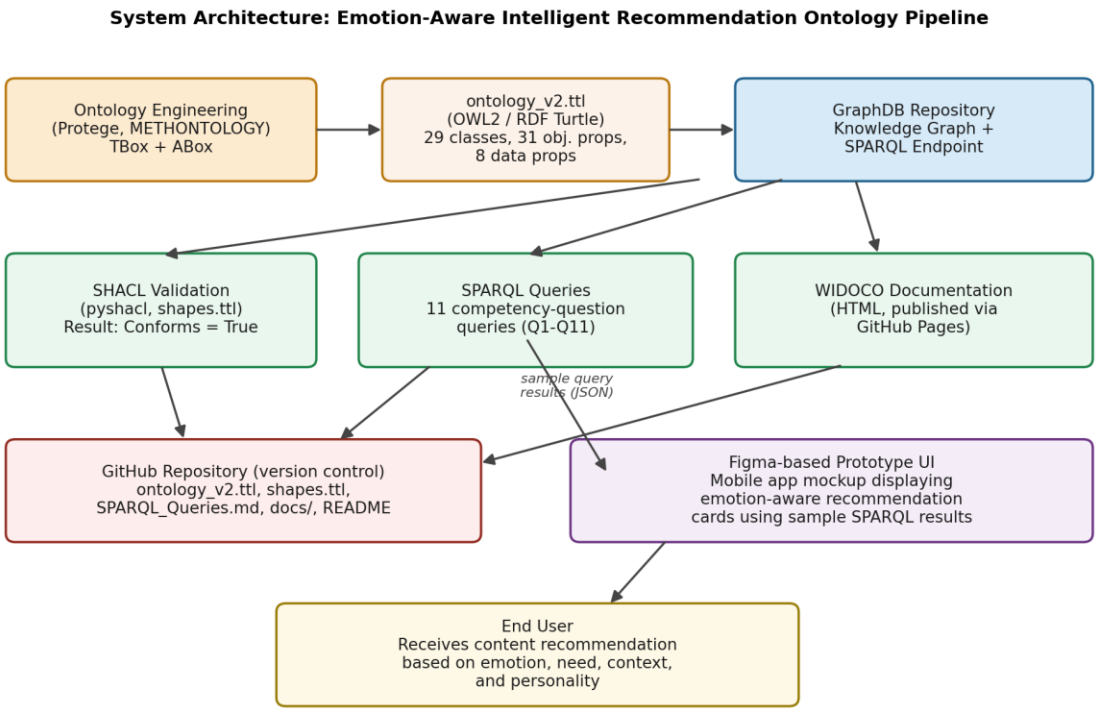

Figure 5. System architecture: from ontology engineering (Protégé, ontology_v2.ttl) through the GraphDB knowledge graph to SHACL validation, SPARQL competency queries, WIDOCO documentation, version control, and the Figma Prototype interface.

The Figma based prototype interface is a front-end mock-up that demonstrates how the recommendations encoded in the knowledge graph could be surfaced to an end user. It is important to clarify the current scope of this prototype: it is not connected to a live GraphDB SPARQL endpoint or to any real-time external API (such as the Spotify Web API). Instead, the interface consumes a small, static JSON file containing the results of representative SPARQL queries (in particular, the Emotion → Need → Content chain returned by Q5 and the platform-availability information returned by Q9), and renders this data as "recommendation cards" — each card displaying the user's name, their current emotion, the need it creates, the recommended content item, its category and duration, the platform on which it is available, and the recommendation's confidence score (ex:hasConfidenceScore). This design choice allows the prototype to demonstrate the ontology-driven recommendation workflow end-to-end (knowledge graph → SPARQL query → user-facing recommendation) without requiring a live backend integration, which is left as future work (see Section 8 and Section 10).

The prototype was designed in Figma as a mobile application interface demonstrating the practical use of the ontology. Users can create an account, log in, view their emotional state, receive personalized recommendations, track interaction history, and manage their profile information. The interface was designed to simulate a real-world emotion-aware recommendation system while keeping Semantic Web technologies in the background.

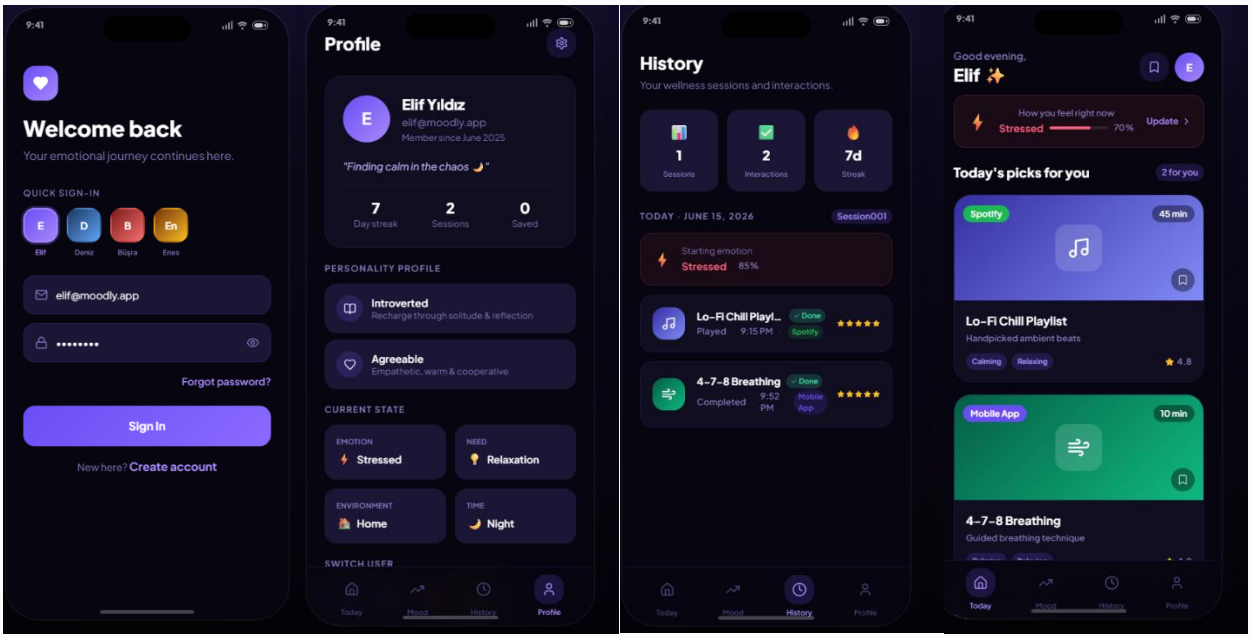

Figure 6. Figma-based mobile prototype demonstrating ontology-driven emotion-aware recommendations, user profile management, interaction history, and personalized content delivery.

6. SPARQL Queries

Eleven SPARQL queries were implemented and executed against the GraphDB repository to demonstrate the functionality of the knowledge graph and to directly answer the competency questions defined in Section 2.3. The full set of queries, together with their expected results, is also maintained in the project repository as SPARQL_Queries.md. Table 10 categorizes the eleven queries into four types — basic retrieval, reasoning (class-hierarchy / inferred-type), aggregation, and temporal / provenance — following the categorization required for this report.

Query	Type	Purpose
Q1	Basic retrieval	Repository sanity check — confirms that ontology triples were loaded into GraphDB.
Q2	Basic retrieval	Retrieves each user and the emotion they currently feel.
Q3	Reasoning (class hierarchy)	Retrieves only users whose emotion is typed as a sub-class of ex:NegativeEmotion.
Q4	Reasoning (class hierarchy)	Retrieves content suitable for a user based on their personality sub-class.
Q5	Reasoning (property chain)	Demonstrates the core Emotion → Need → Content recommendation chain.

Query	Type	Purpose
Q6	Aggregation	Counts how many users fall into each top-level emotion category, using GROUP BY and COUNT.
Q7	Temporal / ordering	Retrieves dynamic emotion observations and their intensity levels, ordered by intensity.
Q8	Basic retrieval (session)	Retrieves the interaction events recorded within an emotion session.
Q9	Basic retrieval	Retrieves which content items are available on which platforms.
Q10	Provenance retrieval	Retrieves the data source and source URL used to populate user-interaction instances.
Q11	Temporal / reasoning	Tracks emotion change over time via the changesOverTime property.

Table 10. Categorization of the eleven SPARQL queries implemented for this project.

The remainder of this section presents each query in full, together with its purpose, expected result, and a placeholder for the corresponding GraphDB SPARQL Workbench screenshot showing the query and its result table.

Q1 — Repository Sanity Check

Category: Basic retrieval **Competency Question:** Pre-requisite for all CQs (verifies the KG is loaded)

This is the first query run after loading ontology_v2.ttl into the GraphDB repository. It performs an unrestricted triple-pattern match (?s ?p ?o) to confirm that the repository is non-empty and that both TBox and ABox triples were imported successfully before any competency-question testing begins.

```
PREFIX ex:   <http://example.org/emotion-aware-recommendation#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT * WHERE {
  ?s ?p ?o .
}
LIMIT 100
```

Expected result: *The query returns 100 triples (the LIMIT), drawn from across the ontology's class declarations, property declarations, and individuals — confirming successful loading.*

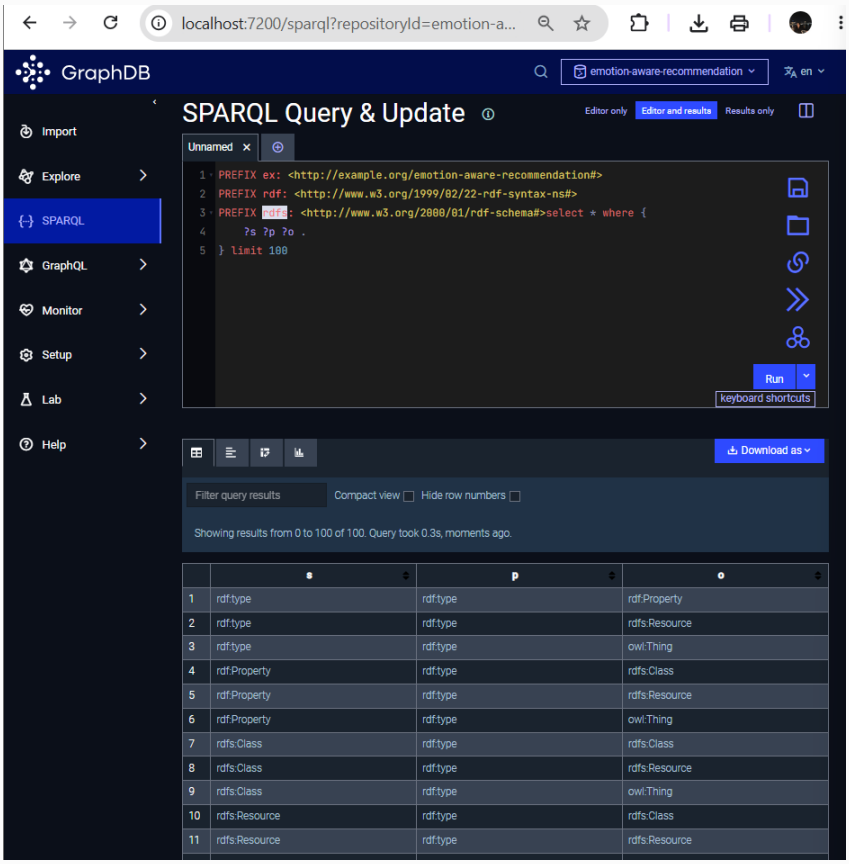

Figure: GraphDB result for Q1.

Q2 — Retrieve Users and Their Emotions

Category: Basic retrieval **Competency Question:** CQ1 — Which emotional state is each user currently experiencing?

This query retrieves every individual typed as ex:User together with the emotion individual connected to it via ex:feels, directly answering CQ1.

```
PREFIX ex:  <http://example.org/emotion-aware-recommendation#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?user ?emotion
WHERE {
  ?user rdf:type ex:User .
  ?user ex:feels ?emotion .
}
```

Expected result: *Elif – Stress, Büşra – Boredom, Enes – Tiredness, Deniz – Sadness (4 rows).*

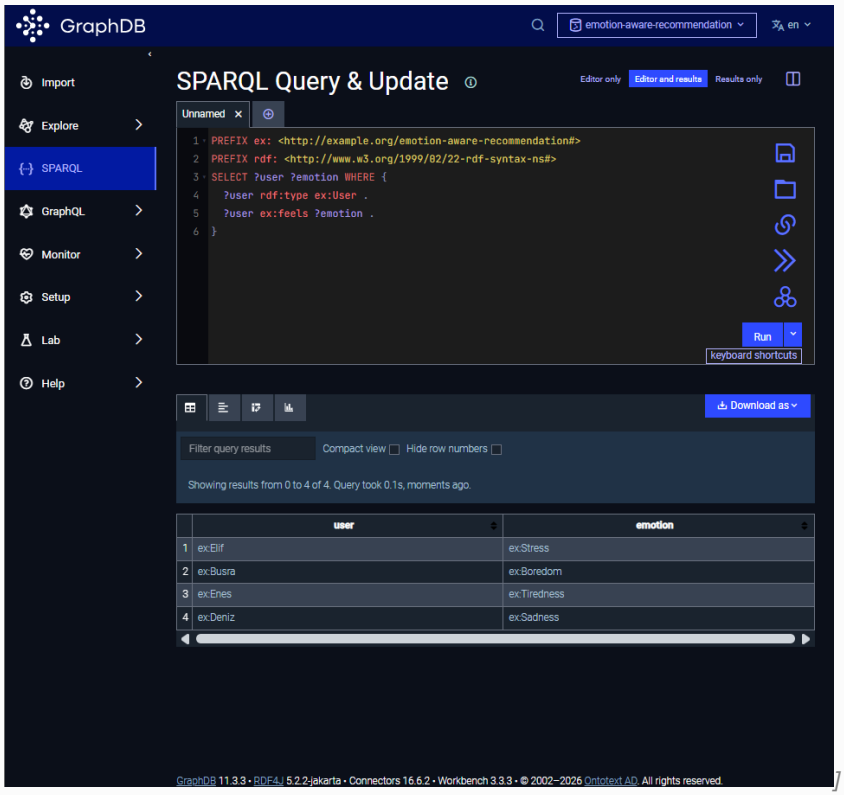

Figure: GraphDB result for Q2.

Q3 — Retrieve Users with Negative Emotions

Category: Reasoning (class hierarchy) **Competency Question:** CQ2 — Which users are currently experiencing a negative emotion?

This query exploits the Emotion sub-class hierarchy described in Section 3.6.2: rather than enumerating Stress, Sadness, Anxiety, and Tiredness individually, it filters on rdf:type ex:NegativeEmotion, so any emotion individual classified under this sub-class — present now or added later — is automatically included.

```
PREFIX ex:  <http://example.org/emotion-aware-recommendation#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?user ?emotion
WHERE {
  ?user ex:feels ?emotion .
  ?emotion rdf:type ex:NegativeEmotion .
}
```

Expected result: *Elif – Stress, Enes – Tiredness, Deniz – Sadness (3 rows; Büşra is excluded because Boredom is typed as ex:NeutralEmotion).*

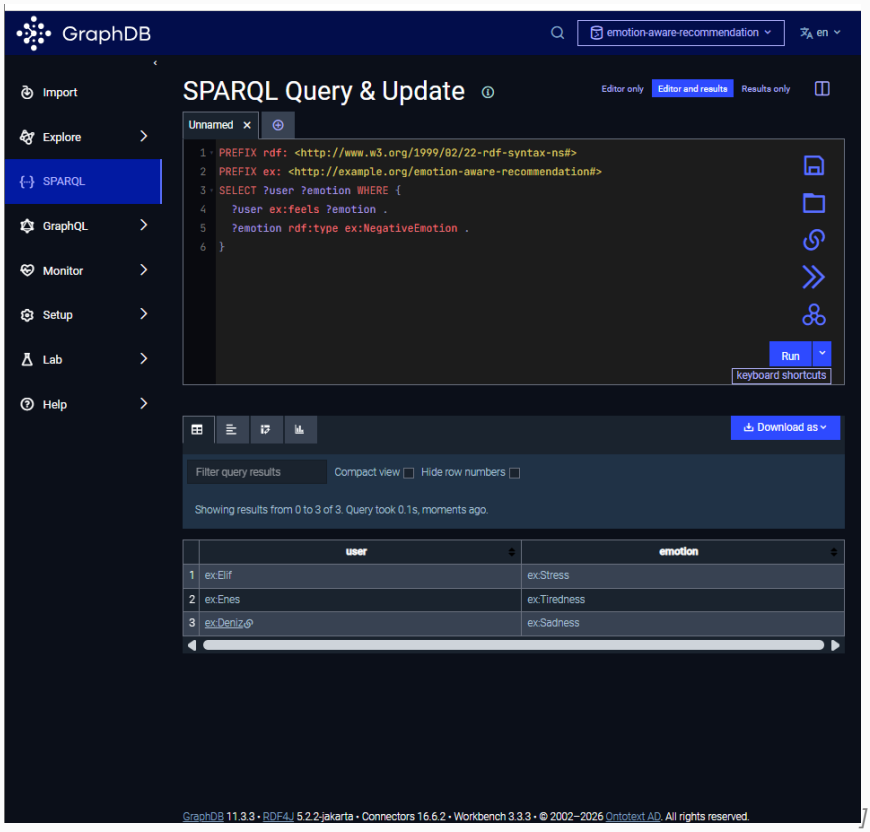

Figure: GraphDB result for Q3.

Q4 — Personality-Based Content Recommendation

Category: Reasoning (class hierarchy) **Competency Question:** CQ3 — Which content items are suitable for a user given their personality type?

This query joins the User → Personality and Content → Personality relationships to recommend content for each user according to their personality individual, directly demonstrating how the Personality sub-class hierarchy (Section 3.3, Table 3) drives content matching.

```
PREFIX ex: <http://example.org/emotion-aware-recommendation#>

SELECT ?user ?personality ?content
WHERE {
  ?user ex:hasPersonality ?personality .
  ?content ex:suitableForPersonality ?personality .
}
```

Expected result: *Introvert users (Elif, Enes) receive ex:LoFiPlaylist and ex:BreathingExercise; extrovert users (Büşra, Deniz) receive ex:ComfortMovie.*

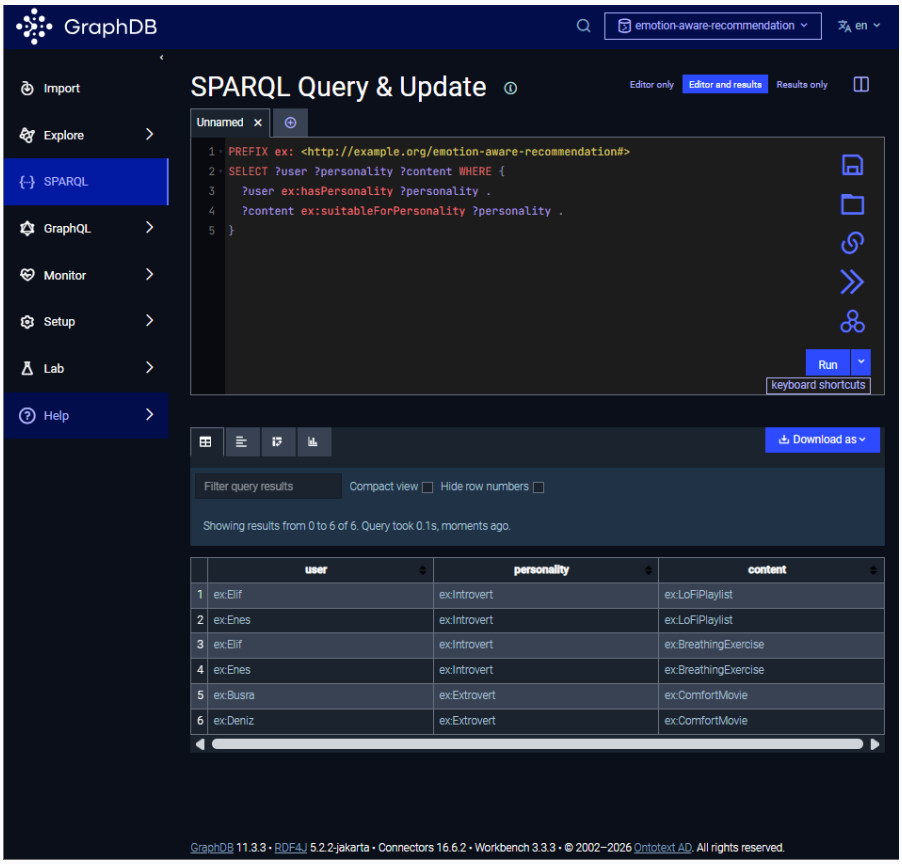

Figure: GraphDB result for Q4.

Q5 — Emotion-Based Need and Content Recommendation

Category: Reasoning (property chain) **Competency Question:** CQ4 — Given a user's emotion, what need does it create and which content satisfies that need?

This is the central competency-question query of the ontology: it chains three object properties (ex:feels, ex:createsNeed, ex:satisfiedBy) across three classes (Emotion, Need, Content) to traverse the full recommendation logic for every user in a single query. The results of this query are precisely the data shown in the recommendation cards of the prototype interface (Section 5.5).

```
PREFIX ex: <http://example.org/emotion-aware-recommendation#>

SELECT ?user ?emotion ?need ?content
WHERE {
  ?user ex:feels ?emotion .
  ?emotion ex:createsNeed ?need .
  ?need ex:satisfiedBy ?content .
}
```

Expected result: *Stress → Relaxation → {LoFiPlaylist, BreathingExercise}; Boredom → Distraction → PuzzleGame; Tiredness → Motivation → MotivationalPodcast; Sadness → EmotionalSupport → ComfortMovie (5 rows, since Relaxation is satisfied by two content items).*

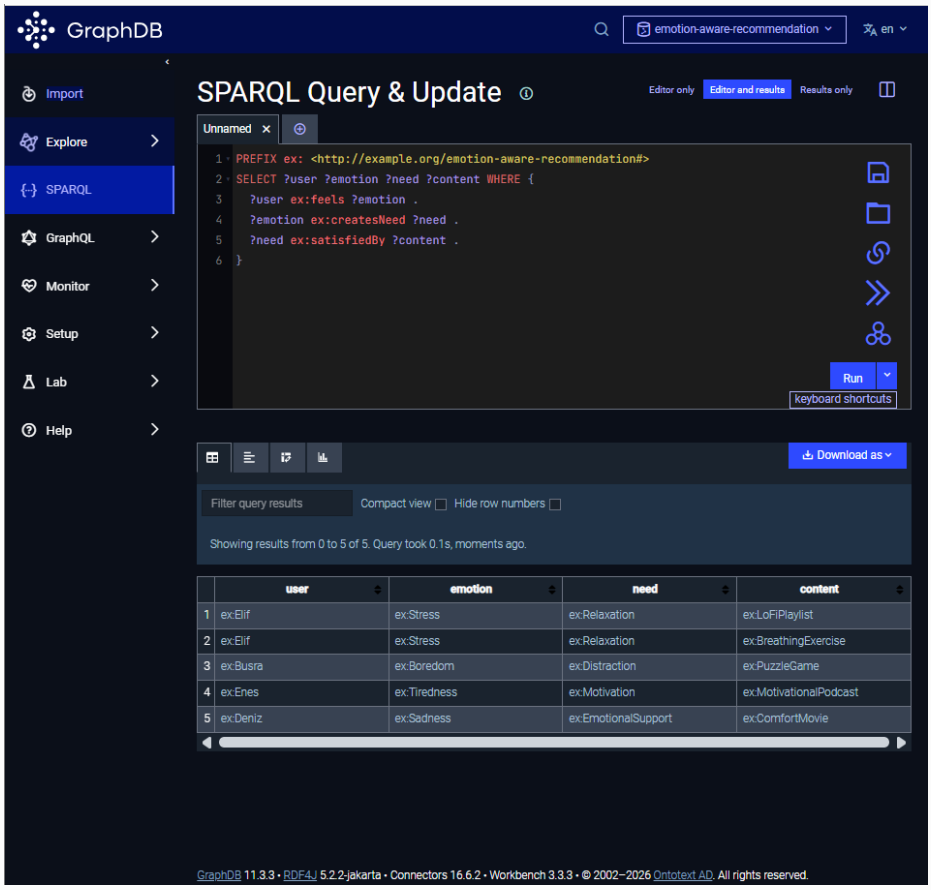

Figure: GraphDB result for Q5.

Q6 — Count Users by Emotion Type

Category: Aggregation **Competency Question:** CQ5 — How many users fall into each high-level emotion category?

This aggregation query uses GROUP BY and the COUNT aggregate function together with a FILTER...IN clause restricted to the three top-level Emotion sub-classes, producing a per-category user count that demonstrates the ontology's ability to support analytical, dashboard-style queries.

```
PREFIX ex:  <http://example.org/emotion-aware-recommendation#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?emotionType (COUNT(?user) AS ?userCount)
WHERE {
  ?user ex:feels ?emotion .
```

```
  ?emotion rdf:type ?emotionType .
  FILTER(?emotionType IN (ex:NegativeEmotion, ex:NeutralEmotion, ex:PositiveEmotion))
}
GROUP BY ?emotionType
```

Expected result: *NegativeEmotion = 3 (Elif, Enes, Deniz); NeutralEmotion = 1 (Büşra); PositiveEmotion is omitted from the result set because no user currently feels a PositiveEmotion individual.*

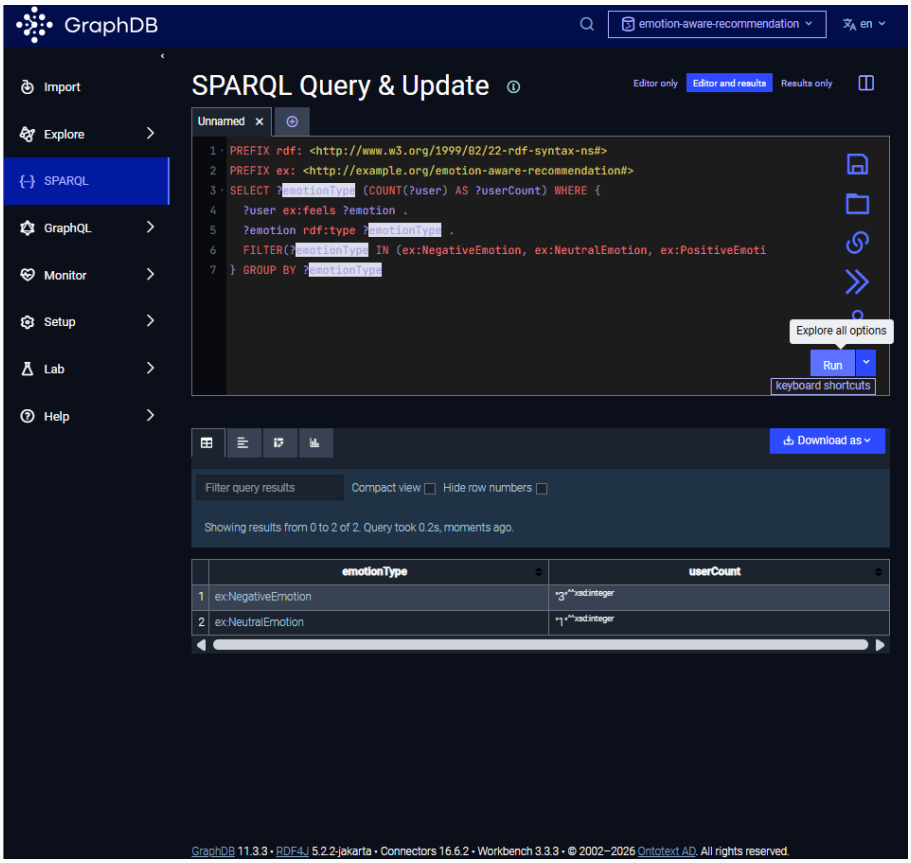

Figure: GraphDB result for Q6.

Q7 — Retrieve Dynamic Emotion Intensity

Category: Temporal / ordering **Competency Question:** CQ6 — How does a user's dynamic emotion intensity evolve, and which observations are most intense?

This query retrieves the Phase 2 DynamicEmotion individuals linked to each user via ex:hasDynamicEmotion, together with their intensity scores (ex:hasIntensityLevel), and orders the results from most to least intense using ORDER BY DESC.

```
PREFIX ex: <http://example.org/emotion-aware-recommendation#>

SELECT ?user ?dynEmotion ?intensity
```

```
WHERE {
  ?user ex:hasDynamicEmotion ?dynEmotion .
  ?dynEmotion ex:hasIntensityLevel ?intensity .
}
ORDER BY DESC(?intensity)
```

Expected result: *ElifStressHigh (Elif) = 0.85, EnesLowMotivationMorning (Enes) = 0.65 (2 rows, ordered by descending intensity).*

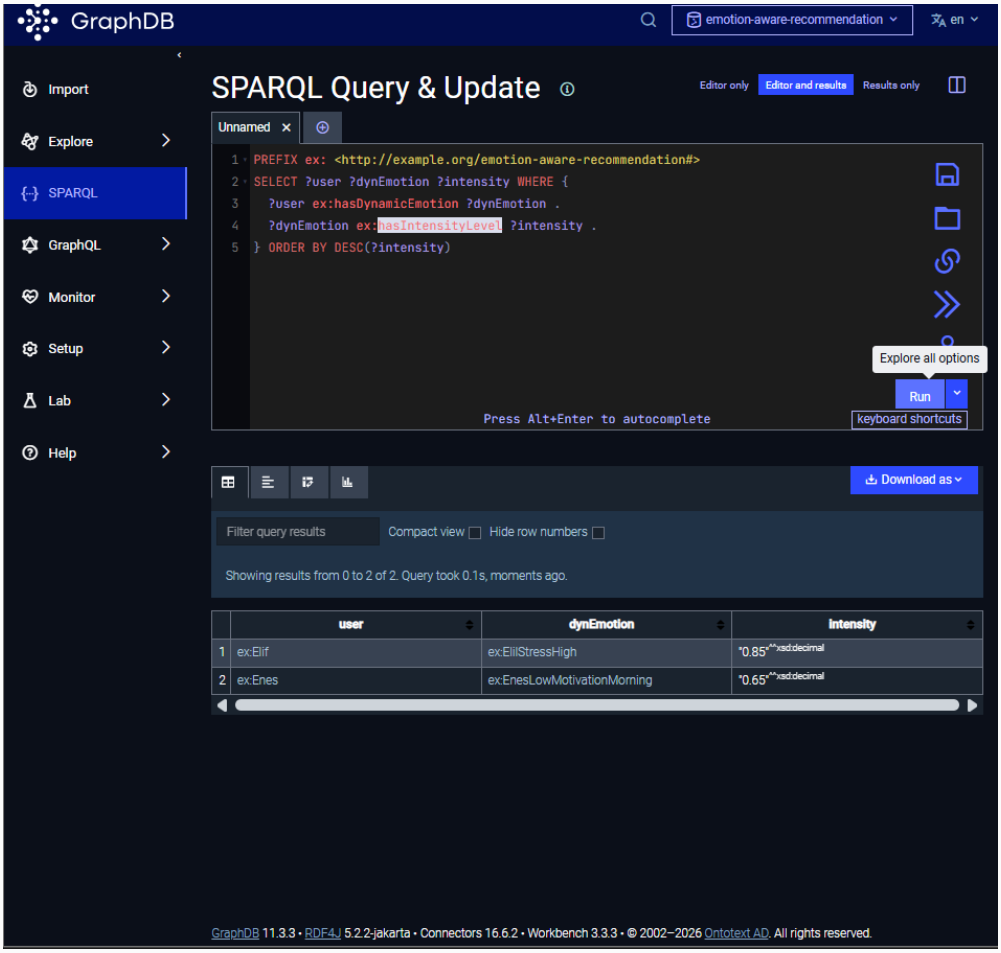

Figure: GraphDB result for Q7.

Q8 — Retrieve Session Interaction Logs

Category: Basic retrieval (session) **Competency Question:** CQ7 — What interaction events occurred with which content during a session?

This query traverses the EmotionSession → UserInteraction → Content chain introduced in the Phase 2 extension (Section 3.6.3 / 3.6.4), retrieving every interaction recorded within a session, the content item it relates to, and its interaction type.

```
PREFIX ex: <http://example.org/emotion-aware-recommendation#>

SELECT ?session ?interaction ?content ?type
WHERE {
  ?session ex:hasInteraction ?interaction .
  ?interaction ex:interactedWith ?content .
  ?interaction ex:hasInteractionType ?type .
}
```

Expected result: *Session001 contains Interaction001 (LoFiPlaylist, type = 'play') and Interaction002 (BreathingExercise, type = 'complete') (2 rows).*

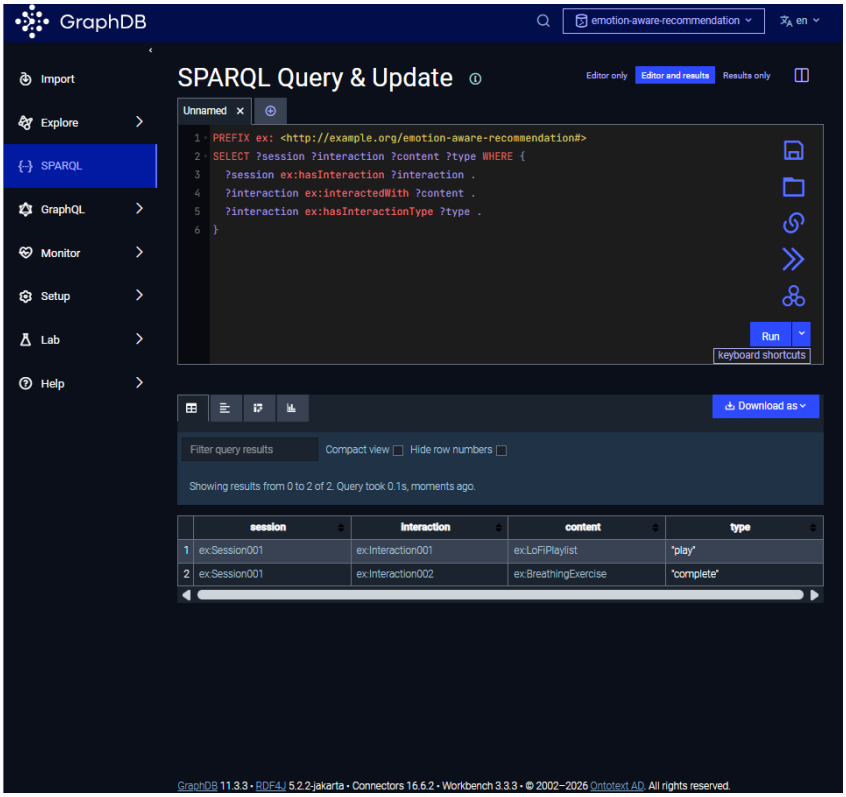

Figure: GraphDB result for Q8.

Q9 — Retrieve Content Platform Availability

Category: Basic retrieval **Competency Question:** CQ8 — On which platforms is a given content item available?

This query retrieves the ex:availableOnPlatform relationship for every content item, which the prototype interface (Section 5.5) uses to display where a recommended item can be accessed.

```
PREFIX ex: <http://example.org/emotion-aware-recommendation#>

SELECT ?content ?platform
```

```
WHERE {
  ?content ex:availableOnPlatform ?platform .
}
```

Expected result: *LoFiPlaylist and MotivationalPodcast → Spotify; ComfortMovie → Netflix; BreathingExercise, MeditationAudio, and PuzzleGame → MobileApp (6 rows).*

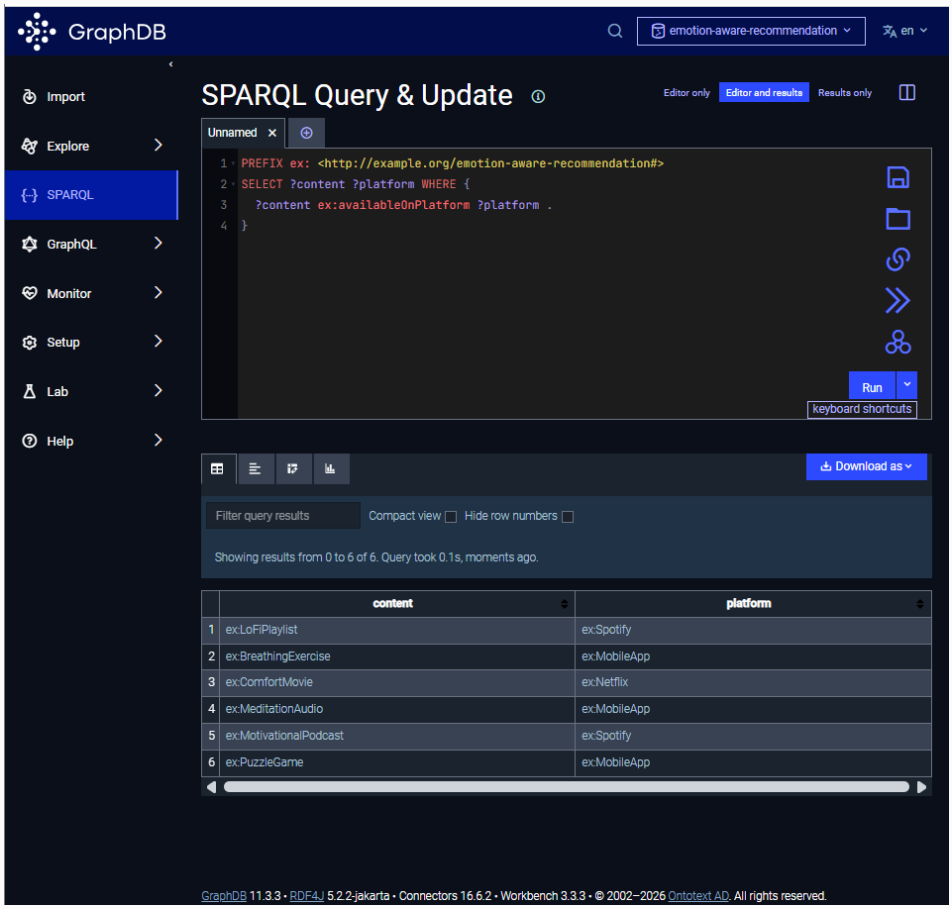

Figure: GraphDB result for Q9.

Q10 — Retrieve Data Provenance for Interactions

Category: Provenance retrieval **Competency Question:** CQ8 — What is the provenance (data source) of a given interaction record?

This query traverses the provenance chain introduced in Section 3.6.4: each UserInteraction is linked via ex:populatedFrom to a DataSource individual, which in turn carries an ex:hasSourceURL literal documenting where the data originated.

```
PREFIX ex: <http://example.org/emotion-aware-recommendation#>

SELECT ?interaction ?source ?url
```

```
WHERE {
  ?interaction ex:populatedFrom ?source .
  ?source ex:hasSourceURL ?url .
}
```

Expected result: *Interaction001 and Interaction002 are both populated from SpotifyAPISource, with source URL https://developer.spotify.com/documentation/web-api (2 rows).*

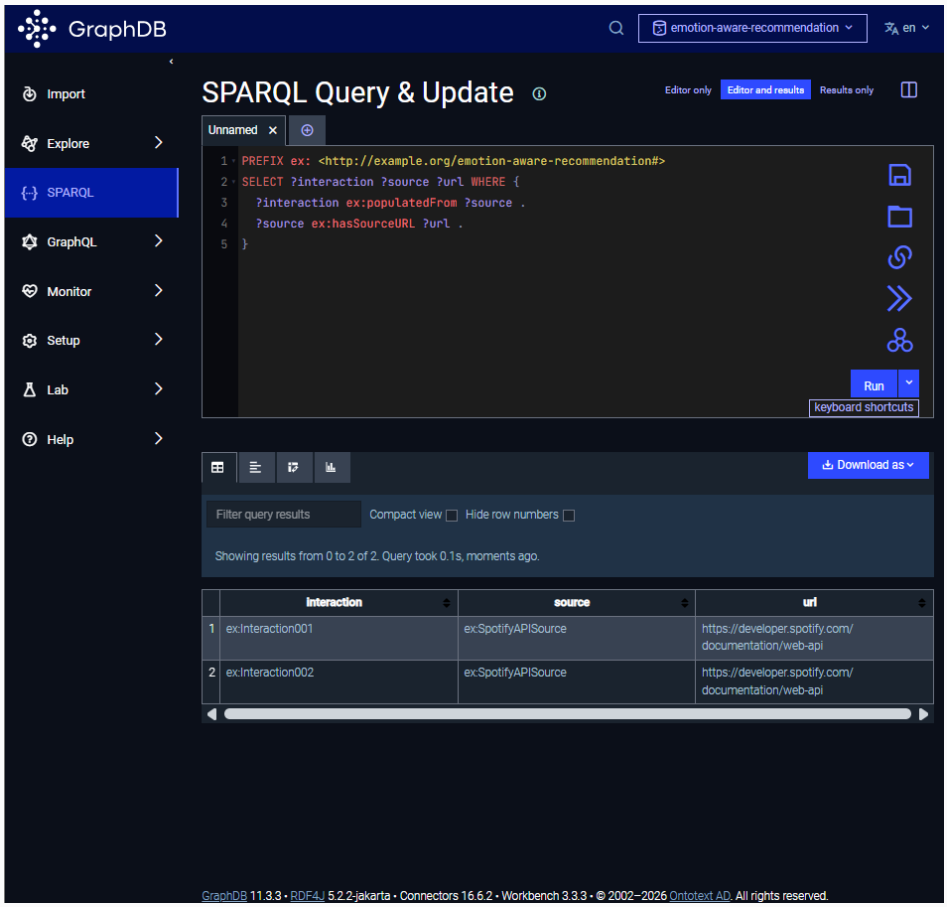

Figure: GraphDB result for Q10.

Q11 — Track Emotion Change Over Time

Category: Temporal / reasoning **Competency Question:** CQ6 — How does a user's dynamic emotion intensity evolve over time?

This query retrieves the temporal-transition relationship introduced by the changesOverTime shortcut property (Section 3.6.3), demonstrating that the knowledge graph can represent and query sequences of emotion observations for the same user over time.

```
PREFIX ex: <http://example.org/emotion-aware-recommendation#>
```

```
SELECT ?fromEmotion ?toEmotion
WHERE {
  ?fromEmotion ex:changesOverTime ?toEmotion .
}
```

Expected result: *ElifStressLow ex:changesOverTime ElifStressHigh (1 row), representing Elif's stress level decreasing from 0.85 to 0.30 after listening to the recommended content.*

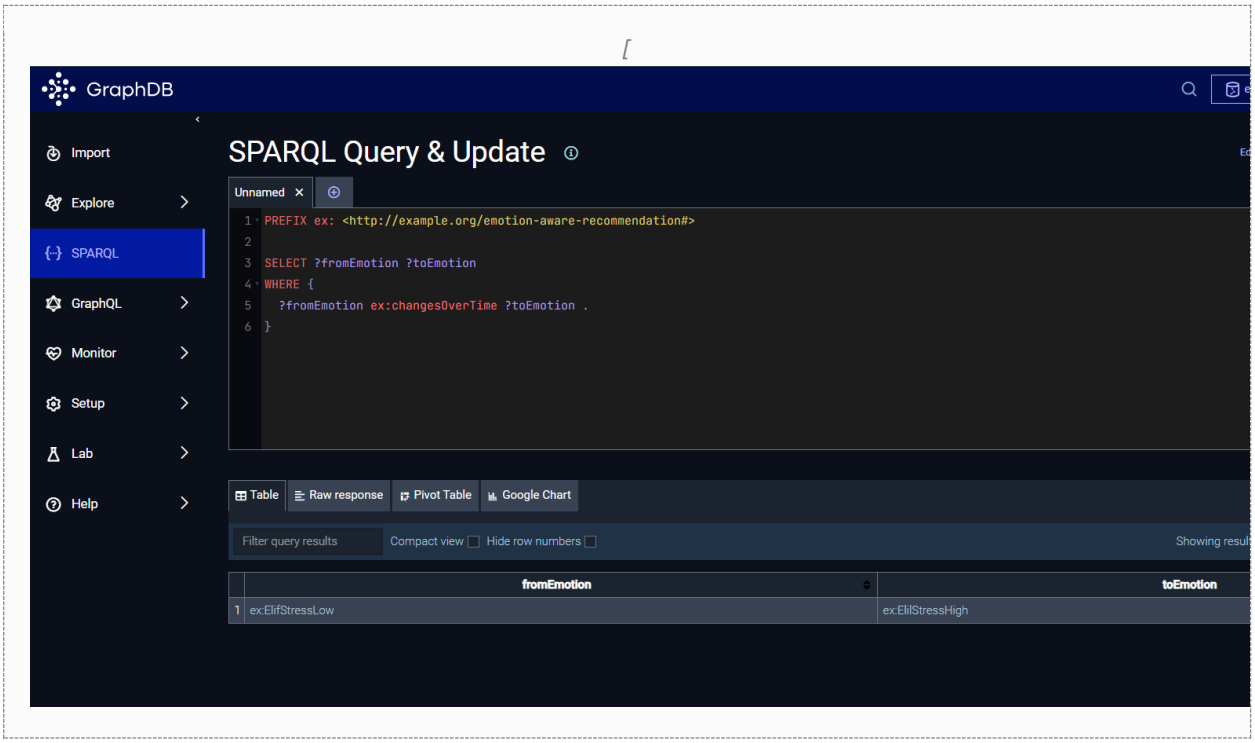

Figure: GraphDB result for Q11.

7. Validation

To ensure the structural integrity and data quality of the knowledge graph, six SHACL (Shapes Constraint Language) node shapes were defined in shapes.ttl, targeting the most data-sensitive classes introduced or extended in the ontology: User, DynamicEmotion, EmotionSession, UserInteraction, Recommendation, and DataSource. SHACL was chosen over OWL-based integrity constraints because it provides a declarative, closed-world style of validation that is well suited to checking that ABox individuals carry the mandatory properties, value ranges, and enumerations required by the application logic — constraints that OWL's open-world semantics cannot enforce directly.

7.1 SHACL Shapes Overview

Table 11 summarizes the six SHACL node shapes, the property each constrains, the type of constraint applied, and the validation message returned when the constraint is violated.

Shape	Target Class	Constrained Property	Constraint	Validation Message
ex:UserShape	ex:User	ex:feels	sh:minCount 1	Each user must have at least one emotional state.
ex:UserShape	ex:User	ex:hasPersonality	sh:minCount 1	Each user must have a personality profile.
ex:DynamicEmotionShape	ex:DynamicEmotion	ex:hasIntensityLevel	sh:datatype xsd:decimal; sh:minInclusive 0.0; sh:maxInclusive 1.0; sh:minCount 1	Dynamic emotion intensity must be a decimal value between 0 and 1.
ex:EmotionSessionShape	ex:EmotionSession	ex:hasTimestamp	sh:datatype xsd:dateTime; sh:minCount 1	Each emotion session must have a timestamp.
ex:EmotionSessionShape	ex:EmotionSession	ex:hasInteraction	sh:minCount 1	Each emotion session must include at least one user interaction.
ex:UserInteractionShape	ex:UserInteraction	ex:interactedWith	sh:class ex:Content; sh:minCount 1	Each user interaction must be linked to a content item.
ex:UserInteractionShape	ex:UserInteraction	ex:hasInteractionType	sh:datatype xsd:string; sh:in (“play” “skip” “complete”); sh:minCount 1	Interaction type must be play, skip, or complete.
ex:RecommendationShape	ex:Recommendation	rdfs:label	sh:minCount 1	Each recommendation should have a readable label.
ex:DataSourceShape	ex:DataSource	ex:hasSourceURL	sh:nodeKind sh:IRI; sh:minCount 1	Each data source must have a source URL.

Table 11. SHACL node shapes, constrained properties, constraints, and validation messages (shapes.ttl).

7.2 Validation Methodology

Validation was performed in Python using the pyshacl library together with rdflib. Listing 2 shows the validation script (validate_shacl.py): the ontology data graph (ontology_v2.ttl) and the SHACL shapes graph (shapes.ttl) are each loaded as separate rdflib Graph objects, and pyshacl.validate() is invoked with inference="rdfs", which applies RDFS entailment (e.g., sub-class membership) to the data graph before checking the shapes — ensuring that, for instance, an individual typed only via a sub-class is still correctly matched by a shape whose sh:targetClass is the corresponding super-class.

```
from pyshacl import validate
from rdflib import Graph

data_graph = Graph()
data_graph.parse("ontology_v2.ttl", format="turtle")

shacl_graph = Graph()
shacl_graph.parse("shapes.ttl", format="turtle")

conforms, results_graph, results_text = validate(
```

```
    data_graph,
    shacl_graph=shacl_graph,
    inference="rdfs",
    debug=False
)

print("Conforms:", conforms)
print(results_text)
```

Listing 2. SHACL validation script (validate_shacl.py), using pyshacl with RDFS inference enabled.

7.3 Validation Results

Figure 7 shows the terminal output of validate_shacl.py executed against the published ontology file and shapes.ttl. The script prints Conforms: True from the boolean conforms return value, and the pyshacl validation report (results_text) independently confirms Conforms: True with zero constraint violations across all nine property constraints defined in the six shapes.

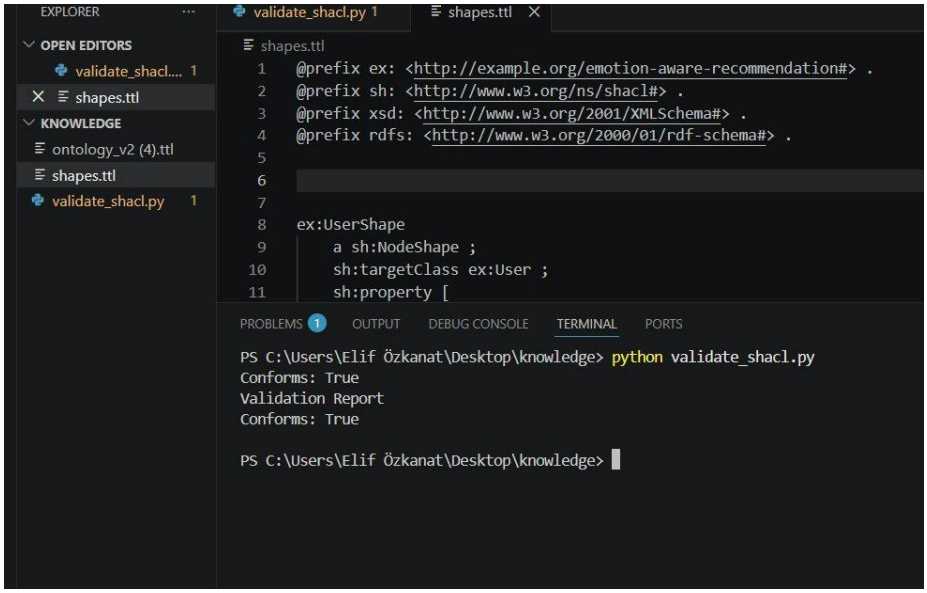

Figure 7. Terminal output of validate_shacl.py: Conforms: True, confirming that the ABox satisfies all SHACL shapes.

This result demonstrates that: (1) every User individual (Elif, Büsra, Enes, Deniz) has at least one ex:feels and one ex:hasPersonality assertion; (2) every DynamicEmotion individual (ElifStressHigh, ElifStressLow, EnesLowMotivationMorning) carries an ex:hasIntensityLevel value that is a decimal in the range [0.0, 1.0]; (3) the EmotionSession individual Session001 has both a valid xsd:dateTime timestamp and at least one ex:hasInteraction link; (4) both UserInteraction individuals are linked to a Content individual and carry an ex:hasInteractionType value drawn from the enumerated set {“play”, “skip”, “complete”}; (5) all four Recommendation individuals carry an rdfs:label; and (6) both DataSource individuals carry an ex:hasSourceURL value that is a well-formed IRI.

7.4 Discussion of Validation Results

The Conforms: True result reflects the fact that the ABox was authored with these constraints in mind from the outset (Section 4.1), rather than retrofitted to satisfy them afterward; nevertheless, the validation pass is meaningful because it was re-run after every structural change to the ontology during Phase 2 development (e.g., after introducing the DynamicEmotion, EmotionSession, UserInteraction, and DataSource classes and their associated shapes), catching potential omissions early. For example, during development the sh:in constraint on ex:hasInteractionType (restricting values to “play”, “skip”, or “complete”) would immediately flag any future automatically-populated UserInteraction individual whose interaction-type label did not match the controlled vocabulary — directly supporting the data-quality goals of the planned LLM-based population pipeline described in Section 8.

One limitation of the current validation suite is its scope: SHACL shapes were defined only for the six classes most relevant to the Phase 2 extension and to the recommendation-justification pattern (Recommendation). Classes such as Content, Effect, and Need do not yet have dedicated shapes (e.g., enforcing that every Content individual has at least one ex:hasEffect and one ex:availableOnPlatform assertion). Extending shape coverage to these classes, and adding shapes that check referential consistency across the Emotion → Need → Content → Effect chain (e.g., that every Need reachable via ex:createsNeed is also reachable via ex:satisfiedBy from at least one Content individual), is identified as a concrete improvement in Section 10.

8. LLM Integration (Planned Future Work)

Large Language Model (LLM) integration was scoped for this project as a planned extension rather than as a component implemented and evaluated in the current submission. This section describes the intended system pipeline, prompt design, hallucination-mitigation strategy, and a design-level illustrative example of the intended interaction, all of which were used to inform ontology design decisions (Section 3.6.3 and 3.6.4) but have not yet been built or executed against a live LLM.

8.1 Planned Pipeline Overview

Two complementary LLM integration paths were designed for future implementation:

- Natural-Language-to-SPARQL Question Answering: an end user asks a question in natural language (e.g., “I feel stressed tonight, what should I listen to?”). An LLM, prompted with the ontology's class and property schema (Section 3.3–3.5) and a small set of example question/SPARQL pairs (few-shot prompting), translates the question into a SPARQL query structurally similar to Q5 (Section 6), optionally with additional FILTER clauses derived from the question (e.g., restricting ex:hasTimeContext to ex:Night). The generated query is executed against the GraphDB SPARQL endpoint, and the result set is used to produce a grounded natural-language answer.

- LLM-Assisted Ontology Population: following the three-stage pipeline of Norouzi et al. (2025) — data preprocessing, text retrieval (e.g., via retrieval-augmented generation over user-interaction logs), and schema-guided triple extraction — an LLM would be prompted with the ontology schema (in particular the UserInteraction, EmotionSession, and DataSource classes and their properties, Section 3.3) and a batch of raw interaction-log text, and asked to emit RDF triples conforming to that schema. Newly emitted triples would then be checked against the SHACL shapes of Section 7 before being merged into the knowledge graph.

8.2 Prompt Design (Planned)

For the natural-language-to-SPARQL path, the planned system prompt would include: (1) the ex: namespace declaration and the relevant subset of class and property definitions from Tables 3–6 (Section 3), expressed as short rdfs:comment-style descriptions; (2) two or three worked examples pairing a natural-language question with its corresponding SPARQL query (drawn from the competency questions in Table 1 and the queries in Section 6); and (3) an explicit instruction that the model must only use classes, properties, and prefixes that appear in the supplied schema, and must return a single, syntactically valid SPARQL SELECT query with no additional commentary. For the ontology-population path, the planned prompt would additionally include the SHACL shapes from shapes.ttl as constraints the emitted triples must satisfy (e.g., ex:hasInteractionType values restricted to “play”, “skip”, or “complete” per the sh:in constraint in Table 11).

8.3 Hallucination Mitigation Strategies (Planned)

Three complementary strategies were identified to reduce hallucination in the planned integration:

- Schema grounding: by supplying only the ontology's actual classes and properties in the prompt and instructing the model to use no others, the search space for the generated SPARQL query is constrained to terms that are guaranteed to exist in the knowledge graph, preventing the model from inventing non-existent predicates.
- Result grounding for answer generation: the natural-language answer presented to the user would be generated only from the SPARQL result set returned by GraphDB (e.g., the rdfs:label values of the Content, Need, and Platform individuals returned by a Q5-style query), rather than from the LLM's own world knowledge — ensuring that any content, platform, or need mentioned in the answer corresponds to an individual that actually exists in the knowledge graph. If the result set is empty, the system would respond that no matching recommendation was found, rather than fabricating one.
- SHACL-gated population: for the ontology-population path, every batch of LLM-emitted triples would be validated with pyshacl against shapes.ttl (Section 7) before being committed to the knowledge graph; triples that violate a shape (e.g., an interaction type outside {“play”, “skip”, “complete”}, or a DynamicEmotion intensity outside [0.0, 1.0]) would be rejected and queued for manual review, directly reusing the validation infrastructure already built and demonstrated in Section 7.

8.4 Illustrative Example Interaction (Design Walkthrough, Not Yet Implemented)

Table 12 walks through a hypothetical end-to-end interaction under the planned pipeline, using the existing ontology data from Section 5.2 as the grounding source. This walkthrough illustrates the intended behavior of the system once implemented; it was produced manually as a design exercise and does not reflect an actual LLM execution.

Step	Description
1. User question	"I feel stressed tonight and I'm home alone — what should I do?"
2. Schema-grounded translation	The LLM, prompted with the schema excerpt for ex:feels, ex:createsNeed, ex:satisfiedBy, ex:suitableForEnvironment, and ex:hasDurationMinutes, generates a SPARQL query structurally equivalent to Q5, additionally filtering on ex:suitableForEnvironment ex:Alone.
3. Query execution	GraphDB executes the generated query against the knowledge graph and returns: emotion = ex:Stress, need = ex:Relaxation, content = {ex:LoFiPlaylist, ex:BreathingExercise}, with ex:LoFiPlaylist additionally satisfying ex:suitableForEnvironment ex:Alone.
4. Grounded answer generation	The LLM composes a natural-language answer using only the labels returned in step 3: "Since you're feeling stressed, a Lo-Fi Playlist (25 minutes, on Spotify) or a 5-minute Breathing Exercise could help you relax — both are well-suited to a quiet evening at home alone."
5. Hallucination check	Both ex:LoFiPlaylist and ex:BreathingExercise exist as ex:Content individuals with the stated ex:hasDurationMinutes and ex:availableOnPlatform values (Section 5.2), so the answer is fully grounded in the knowledge graph; no individual or value outside the SPARQL result set is mentioned.

Table 12. Design walkthrough of the planned natural-language-to-SPARQL interaction, grounded in existing ABox data.

Implementing and empirically evaluating this pipeline — including measuring SPARQL-generation accuracy, answer faithfulness, and the precision/recall of LLM-assisted ontology population against ground truth (using the cosine-similarity, fuzzy-matching, and Jaro-Winkler metrics identified during the Phase 2 literature review of Norouzi et al., 2025) — is the principal item of future work identified in Section 10.

9. Evaluation, Discussion and Conclusion

9.1 Evaluation of Ontology Quality

Consistency: the ontology was authored in Protégé and checked for logical consistency using the HermiT reasoner; no unsatisfiable classes or contradictory axioms were detected across the 29 classes and 31 object properties. The sub-class hierarchies introduced in Phase 2 (Emotion → {Positive, Negative, Neutral, Dynamic}; Personality → six Big-Five sub-classes) are disjoint by construction — no individual is asserted as a member of two sibling sub-classes — and every domain/range restriction declared in the TBox (Tables 4–6) is respected by the corresponding ABox assertions.

Completeness: the ontology and its instance data were designed specifically so that every one of the eight competency questions defined in Section 2.3 could be answered, and Section 6 demonstrates that all eleven SPARQL queries execute successfully and return the expected results. Every class defined in the TBox (Table 3) has at least one populated individual except ex:PositiveEmotion, which is intentionally left empty as a placeholder for future population (Section 3.6.2).

Correctness: the SHACL validation results in Section 7 (Conforms: True across all six shapes and nine property constraints) provide an automated, repeatable check that the ABox satisfies the structural rules considered most important for downstream use — mandatory emotion and personality assertions for users, valid intensity ranges for dynamic emotions, complete session/interaction/provenance chains, labeled recommendations, and well-formed data-source URIs.

9.2 Evaluation of the Knowledge Graph and SPARQL Queries

All eleven SPARQL queries (Section 6) were executed against the GraphDB repository and returned results matching the expected outputs derived analytically from the ABox (Section 3.2). The queries span four categories — basic retrieval (Q1, Q2, Q8, Q9), class-hierarchy reasoning (Q3, Q4), property-chain reasoning (Q5, Q11), and aggregation (Q6) — together with a temporal/ordering query (Q7) and a provenance query (Q10), demonstrating breadth across the query types required for this report. Because the knowledge graph is small (29 classes, 31 object properties, 8 data properties, and roughly 70 individuals, corresponding to a few hundred triples), all queries execute essentially instantaneously in GraphDB; no performance bottlenecks were observed, and query performance at a larger ABox scale (e.g., hundreds of users and thousands of interaction logs, as would result from real Spotify API ingestion) has not yet been evaluated.

The class-relationship analysis in Section 5.4 (Figure 4, Table 9) further confirms that the knowledge graph is well-connected around its two central classes, ex:Content (60 links) and ex:User (42 links), and that the ex:Recommendation hub pattern (Section 3.6.5) successfully aggregates six distinct relationships per recommendation, enabling Q5-style queries to retrieve a complete recommendation justification in a single graph traversal.

9.3 Effectiveness of the LLM Integration

As described in Section 8, LLM integration was scoped as a design exercise and planned future extension rather than an implemented and evaluated component. The design walkthrough in Table 12 demonstrates that the ontology's schema — in particular the directly-traversable Emotion → Need → Content chain and the shortcut properties such as changesOverTime — is well suited to schema-grounded prompting and result-grounded answer generation, which is the primary criterion by which the design's effectiveness can currently

be assessed. A quantitative evaluation of translation accuracy, answer faithfulness, and population precision/recall against ground truth is identified as the highest-priority future work item (Section 10).

9.4 Limitations

- Scale: the ABox contains approximately 70 individuals and a single EmotionSession with two UserInteraction records, which is sufficient to demonstrate the schema and all competency questions but too small to draw statistical conclusions about recommendation quality or to stress-test query performance.
- Unpopulated classes: ex:PositiveEmotion has no individuals, so positive-emotion-driven recommendation scenarios cannot currently be queried; this is a deliberate placeholder rather than a modeling gap.
- Partial SHACL coverage: shapes were defined for 6 of the 29 classes (Section 7.4); classes central to the recommendation chain itself — Content, Need, and Effect — do not yet have dedicated shapes.
- Static prototype: the Figma-based prototype interface. (Section 5.5) renders sample SPARQL query results from a static JSON file rather than querying GraphDB live, and is not connected to any real-time external API such as the Spotify Web API; the ex:populatedFrom links to ex:SpotifyAPISource document an intended provenance pattern rather than a live integration.
- LLM integration not yet implemented: as discussed in Section 8, the natural-language-to-SPARQL and LLM-assisted population pipelines are designed but not built, so no empirical hallucination-rate or accuracy figures are available for this submission.

9.5 Conclusion

This project delivered a complete OWL 2 / RDF ontology for emotion-aware content recommendation, comprising 29 classes, 31 object properties, and 8 data properties, populated with a representative ABox of approximately 70 individuals and deployed as a queryable knowledge graph in GraphDB. The ontology models the central Emotion → Need → Content → Effect recommendation chain and enriches it with Big-Five personality matching, time and environment context, platform availability, user feedback, and — in its Phase 2 extension — temporally-tracked dynamic emotions, behavior profiling, session-based interaction logging, and data provenance, drawing design inspiration from the UniEmotion ontology (Kim, 2013), the Kim and Lee (2015) movie-recommendation ontology, the GRU-RNN behavioral model of Fernandez et al. (2023), and the LLM-based ontology population approach of Norouzi et al. (2025). Eleven SPARQL queries were implemented and verified, directly answering eight competency questions across basic retrieval, class-hierarchy reasoning, aggregation, temporal, and provenance scenarios. Six SHACL shapes covering nine property constraints were validated with pyshacl, yielding Conforms: True with zero violations. The ontology is documented via WIDOCO and published on GitHub, and a Figma-based prototype illustrates how the resulting recommendations could be presented to an end user.

Through this project, the team gained hands-on experience with the full ontology engineering lifecycle — from competency-question elicitation and METHONTOLOGY-guided conceptualization, through OWL 2 formalization in Protégé, knowledge graph deployment and SPARQL querying in GraphDB, to SHACL-based data-quality validation — as well as with integrating design ideas from published research papers into a coherent, extensible schema. The principal direction for future work is to implement and empirically evaluate the LLM integration pipeline designed in Section 8, alongside the schema and SHACL coverage extensions identified in Sections 4.4, 7.4, and 9.4: populating ex:PositiveEmotion, scaling the ABox via real Spotify Web API ingestion and the GoEmotions dataset, extending SHACL shapes to the Content / Need / Effect chain, and connecting the prototype interface to a live GraphDB SPARQL endpoint.

In addition, a user-centered Figma prototype was developed to demonstrate how ontology-driven recommendations can be presented through a modern mobile application interface. The prototype illustrates user registration, emotion tracking, recommendation delivery, interaction history, and profile management features.

10. References

Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. Scientific American, 284(5), 34–43. https://doi.org/10.1038/scientificamerican0501-34

Fernandez-Lopez, M., Gomez-Perez, A., & Juristo, N. (1997). METHONTOLOGY: From ontological art towards ontological engineering. Proceedings of the AAAI97 Spring Symposium Series on Ontological Engineering, 33–40.

Fernandez, F. M. H., Ramana, T. V., Shabana, M., Kannagi, V., & Nalini, M. (2023). Personalized ontology and deep training tree-based optimal GRU-RNN for prediction of students' behavior. Concurrency and Computation: Practice and Experience, 35(1), e7420. https://doi.org/10.1002/cpe.7420

Garijo, D. (2017). WIDOCO: A wizard for documenting ontologies. In C. d'Amato et al. (Eds.), The Semantic Web – ISWC 2017 (Lecture Notes in Computer Science, Vol. 10588, pp. 94–102). Springer. https://doi.org/10.1007/978-3-319-68204-4_9

Kim, H. H. (2013). A semantically enhanced tag-based music recommendation using emotion ontology. In A. Selamat et al. (Eds.), Intelligent Information and Database Systems – ACIIDS 2013, Part II (Lecture Notes in Artificial Intelligence, Vol. 7803, pp. 119–128). Springer. https://doi.org/10.1007/978-3-642-36543-0_13

Kim, O.-S., & Lee, S.-W. (2015). A movie recommendation method based on emotion ontology. Journal of Korea Multimedia Society, 18(9), 1068–1082. https://doi.org/10.9717/kmms.2015.18.9.1068

Musen, M. A. (2015). The Protégé project: A look back and a look forward. AI Matters, 1(4), 4–12. https://doi.org/10.1145/2757001.2757003

Norouzi, S., Barua, S., Christou, I., Gautam, R., Eells, A., Hitzler, P., & Shimizu, C. (2025). Ontology population using LLMs [Course material]. Kansas State University & Wright State University, Knowledge Engineering and Ontologies, Week 11, 2025–2026 Spring Semester.

Ontotext. (2024). GraphDB documentation. Ontotext. https://graphdb.ontotext.com/documentation/

World Wide Web Consortium. (2012). OWL 2 Web Ontology Language document overview (2nd ed.) [W3C Recommendation]. https://www.w3.org/TR/owl2-overview/

World Wide Web Consortium. (2013). SPARQL 1.1 query language [W3C Recommendation]. https://www.w3.org/TR/sparql11-query/

World Wide Web Consortium. (2014). RDF 1.1 concepts and abstract syntax [W3C Recommendation]. https://www.w3.org/TR/rdf11-concepts/

World Wide Web Consortium. (2014). RDF Schema 1.1 [W3C Recommendation]. https://www.w3.org/TR/rdf-schema/

World Wide Web Consortium. (2017). Shapes Constraint Language (SHACL) [W3C Recommendation]. https://www.w3.org/TR/shacl/

11. Appendix

This appendix contains additional material supporting the project, including a further GraphDB class-hierarchy detail view, links to the GitHub repository and WIDOCO documentation, and supplementary SPARQL result screenshots referenced earlier in the report.

A.1 Additional Class Hierarchy Detail View

Figure A1 shows a further detail view of the GraphDB Class Hierarchy, focused on the Context-related classes (ex:TimeContext, ex:Environment), the Platform class, and the Phase 2 ex:EmotionSession class, complementing Figures 1–3 in Section 3.3.1.

Figure A1. Detail view of the GraphDB Class Hierarchy showing ex:TimeContext, ex:Environment, ex:Platform, and ex:EmotionSession.

A.2 Project Links

WIDOCO Documentation: https://elifozknt.github.io/emotion-aware-recommendation-ontology/index.html

GitHub Repository: https://github.com/elifozknt/emotion-aware-recommendation-ontology

The GitHub repository contains ontology_v2.ttl, shapes.ttl, validate_shacl.py, SPARQL_Queries.md, the docs/ folder generated by WIDOCO, and this report.

A.3 Full SPARQL Query Result Screenshots

Section 6 presents the GraphDB SPARQL Workbench screenshot corresponding to each of the eleven queries (Q1–Q11), showing the query text in the editor pane and the corresponding result table, as captured from the GraphDB Workbench.